

Automatic Speech Recognition II

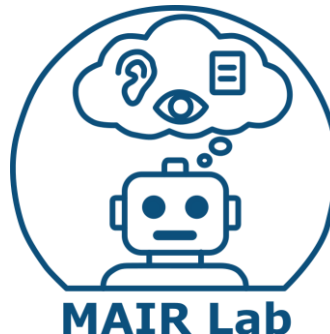
안인규 (Inkyu An)

Speech And Audio Recognition
(오디오 음성인식)

<https://mairlab.kookmin.ac.kr/>



This lecture material refers to
https://github.com/yandexdataschool/speech_course?tab=readme-ov-file and
<https://github.com/markovka17/dla>



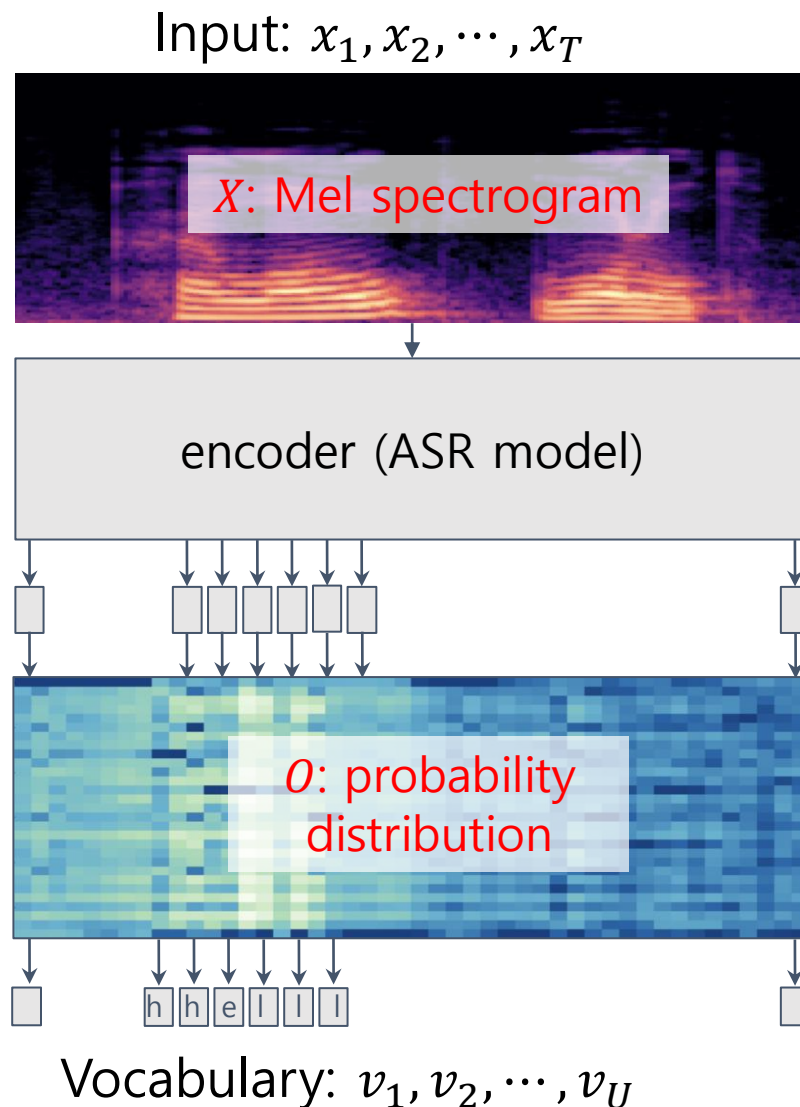
In the previous lecture

- We wanted to build an ASR model
- We have input $x_1, \dots, x_T$ and label $y_1, \dots, y_U$
- Solution:
 - Build a model that predicts distribution over vocabulary: including alphabet (or BPE vocabulary), space, and blank
 - We still don't know the alignment x_t and y_u , but we train the model to maximize all valid paths in the output matrix
 - However, by construction, the model does not have access to its previous predictions when making the next prediction

hell o
target

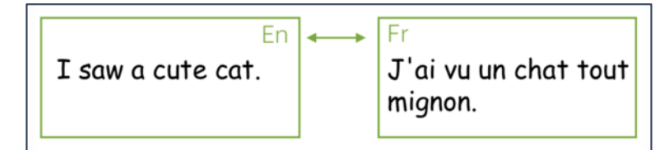
??? ?
What does the ctc
model see?

hell ?
What we want the
model to see

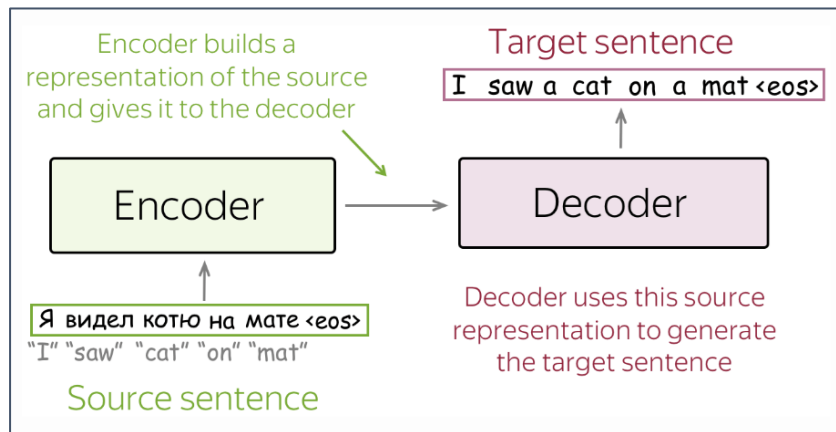


Alignment problem, NLP case

- Task: machine translation
 - We want to transform $x_1, \dots, x_T$ to $y_1, \dots, y_U$ where $T \neq U$ usually

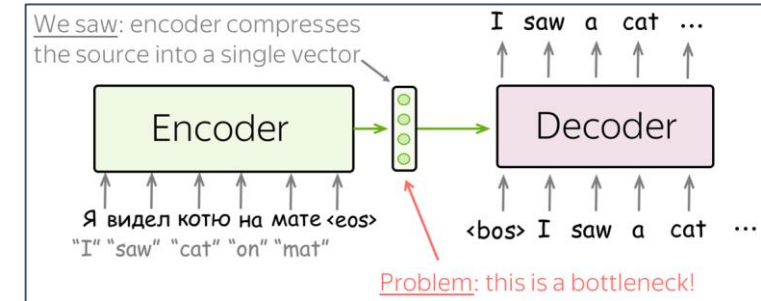


Seq2seq, NeurIPS 2014



Alignment problem, NLP case

- Fixed source representation is suboptimal:
 - For the encoder, it is hard to compress the sentence
 - For the decoder, different information may be relevant at different steps



입력 문장을 하나의 고정된 벡터로만 표현하는 방식은 최적일 수 없음.

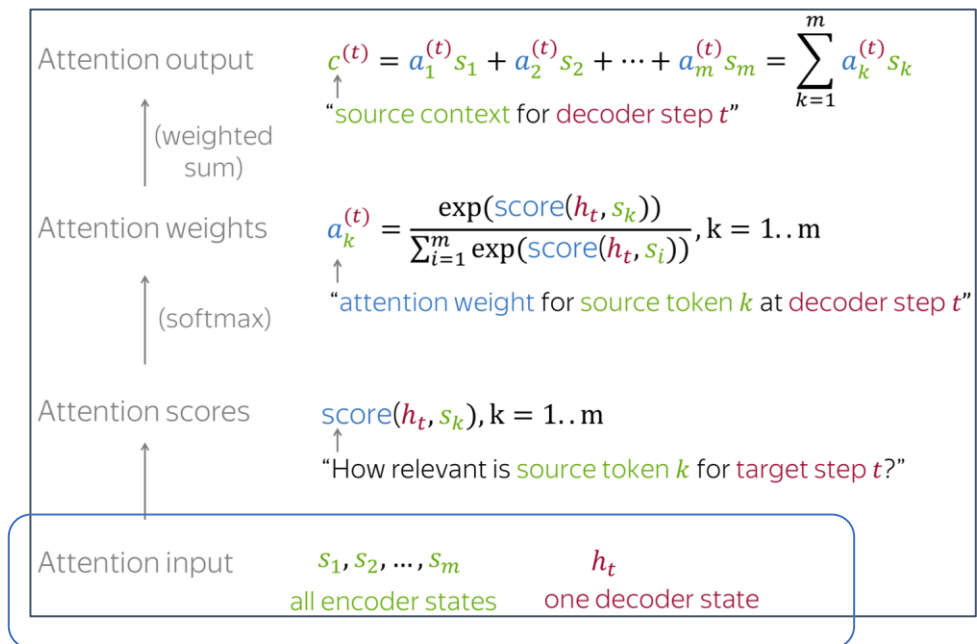
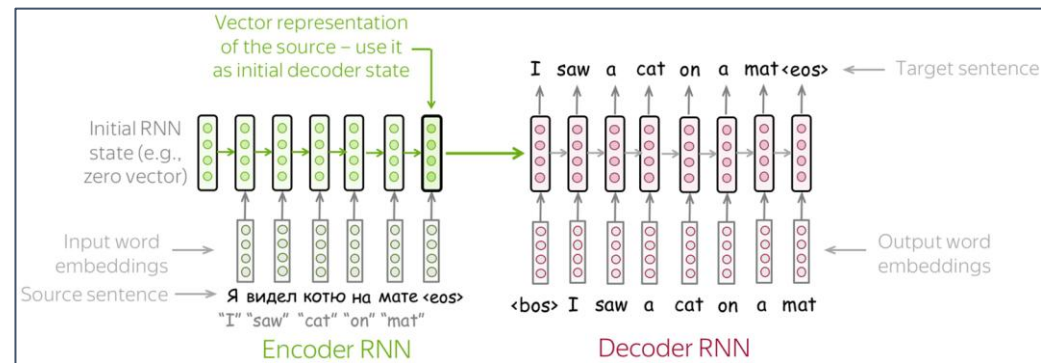
Alignment problem, NLP case

- **Idea**

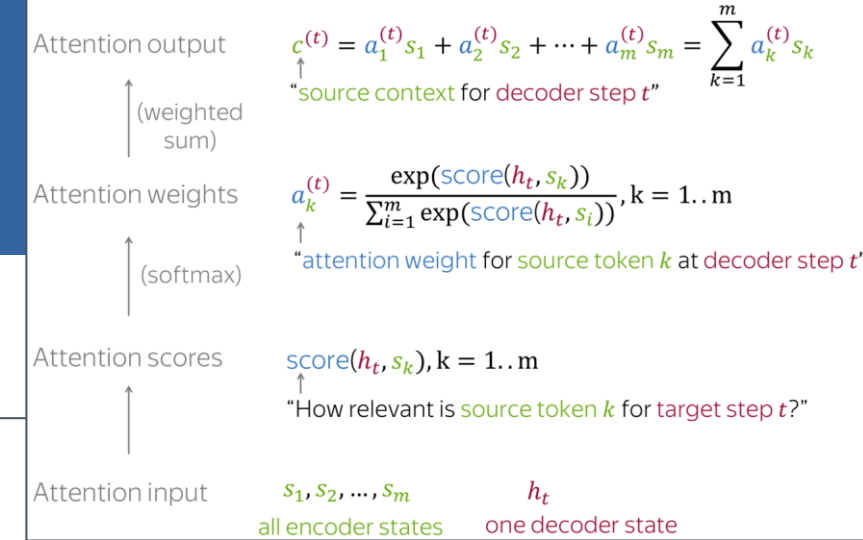
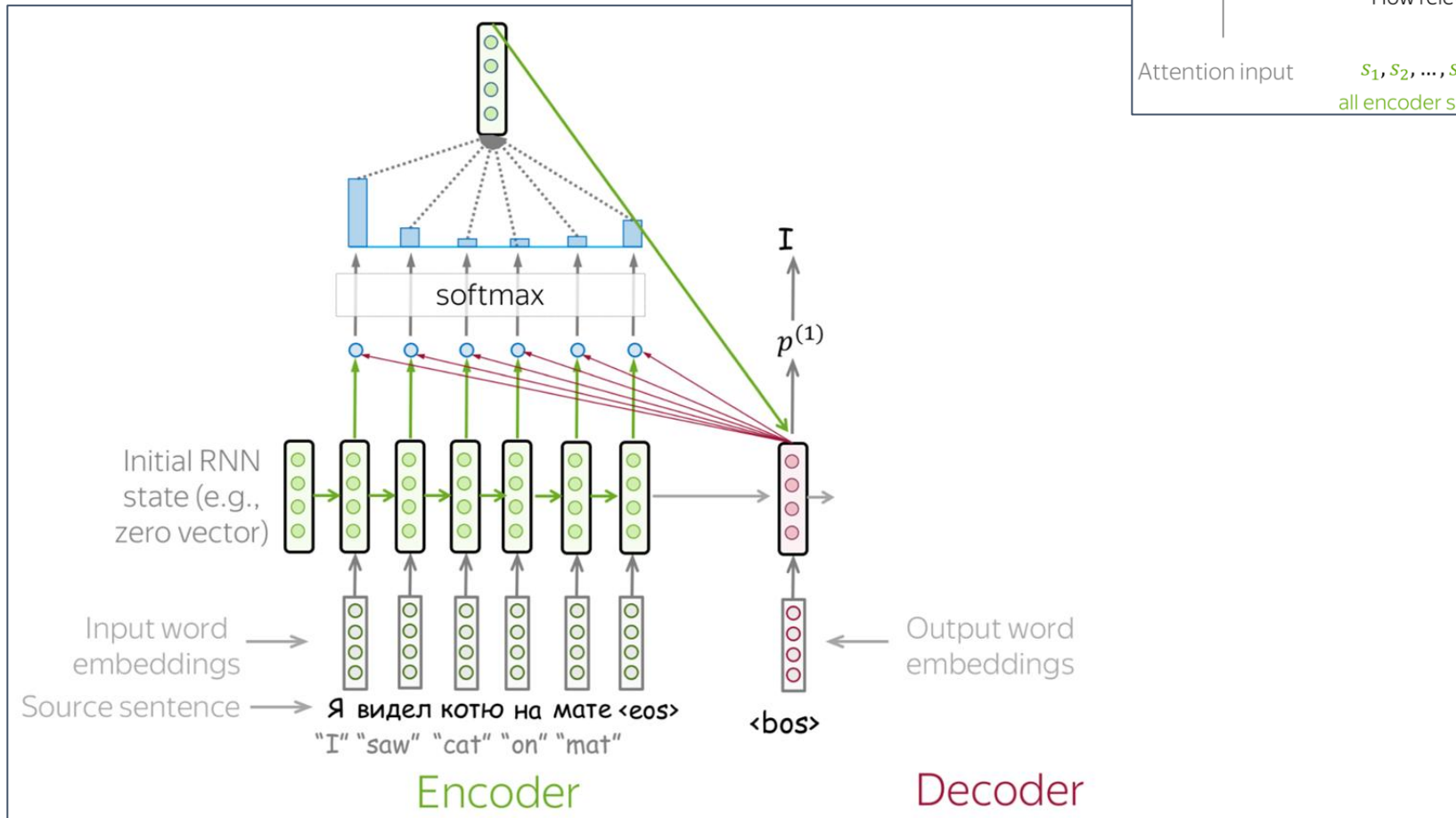
- Use a weighted sum of the original embeddings and recalculate the weights for each decoding step according to previously predicted tokens and the original embeddings

- **Neural Machine Translation by Jointly Learning to Align and Translate**, Dzmitry Bahdanau et. al., 2014

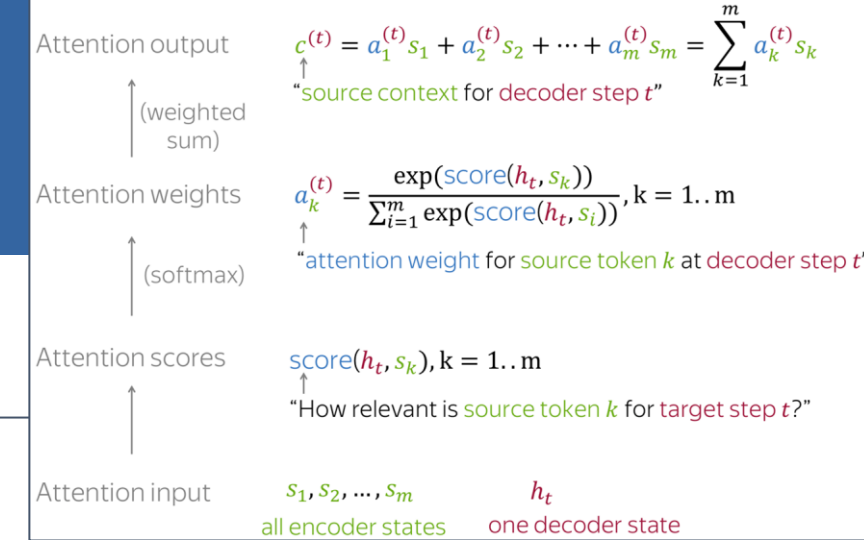
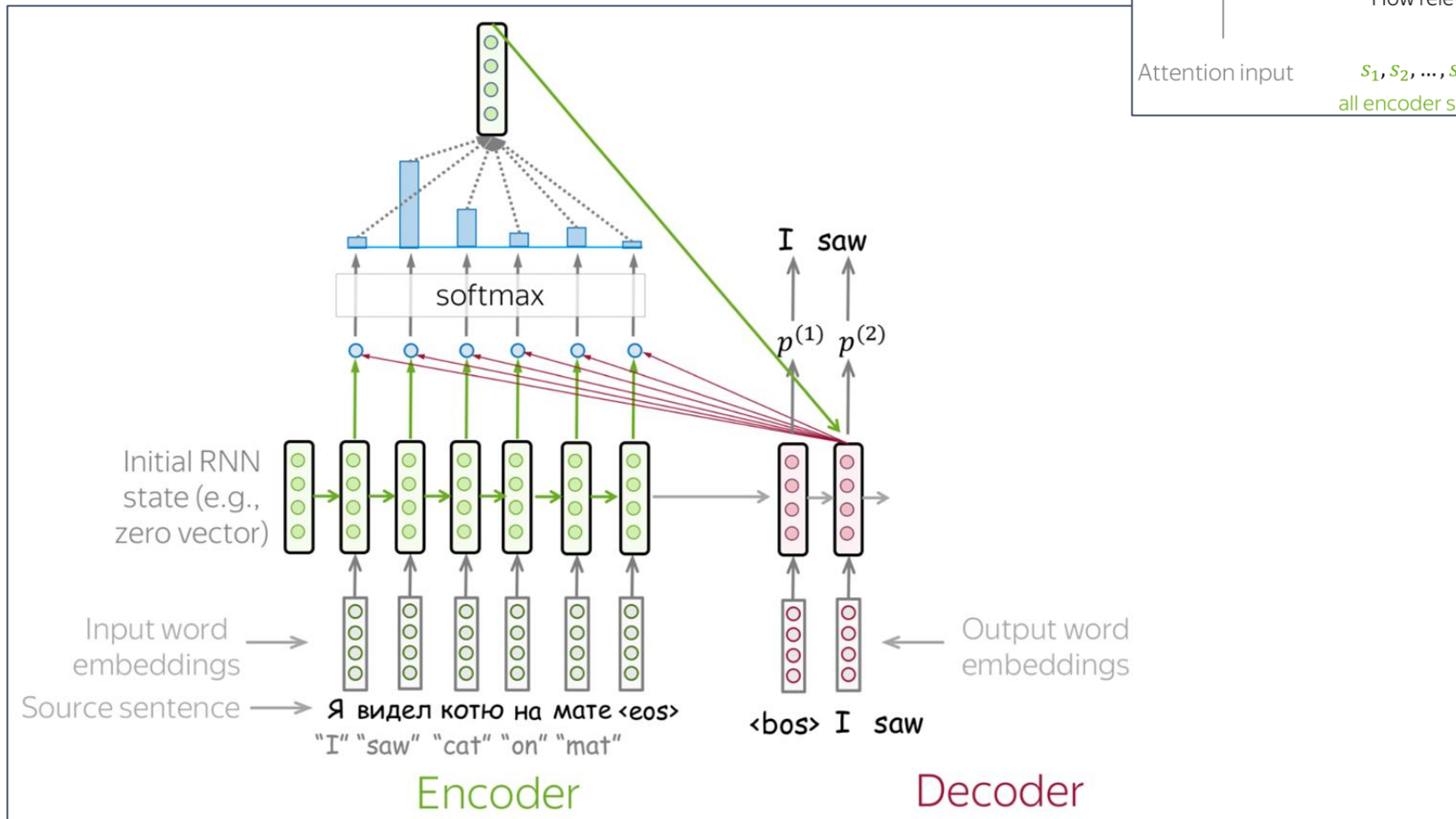
하나의 Output을 출력하기 위해,
모든 입력과의 관계를 고려함.



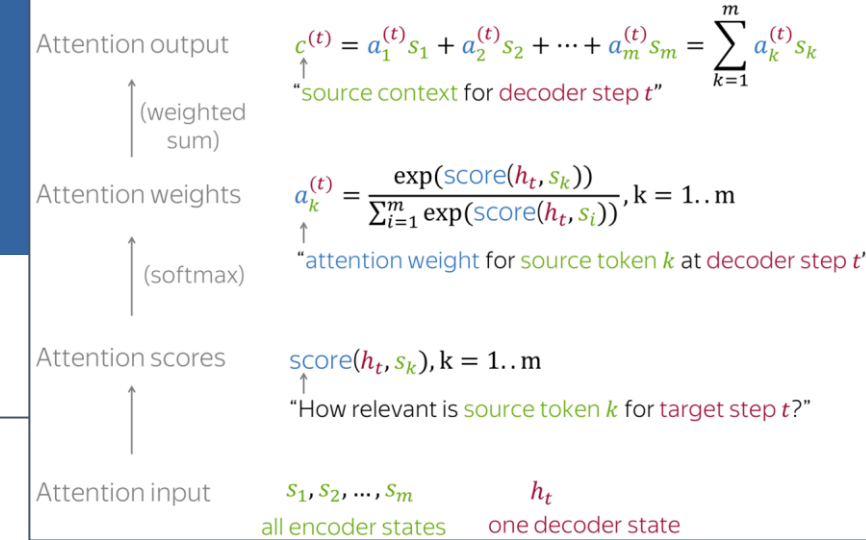
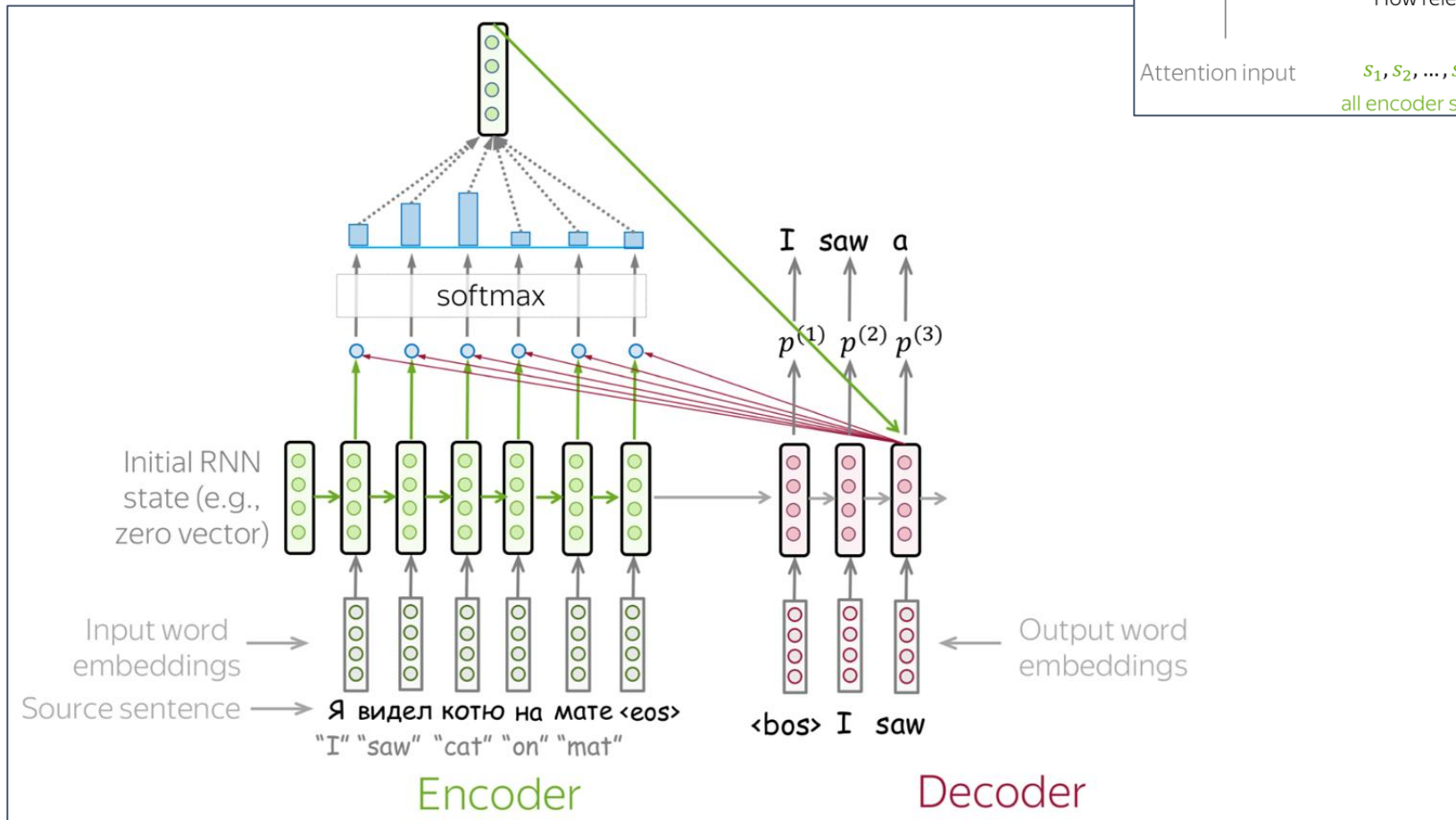
Attention mechanism



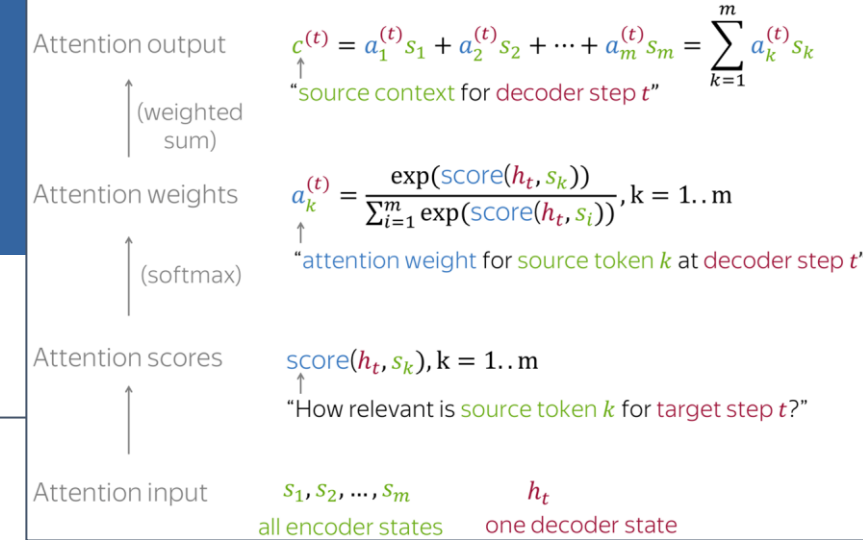
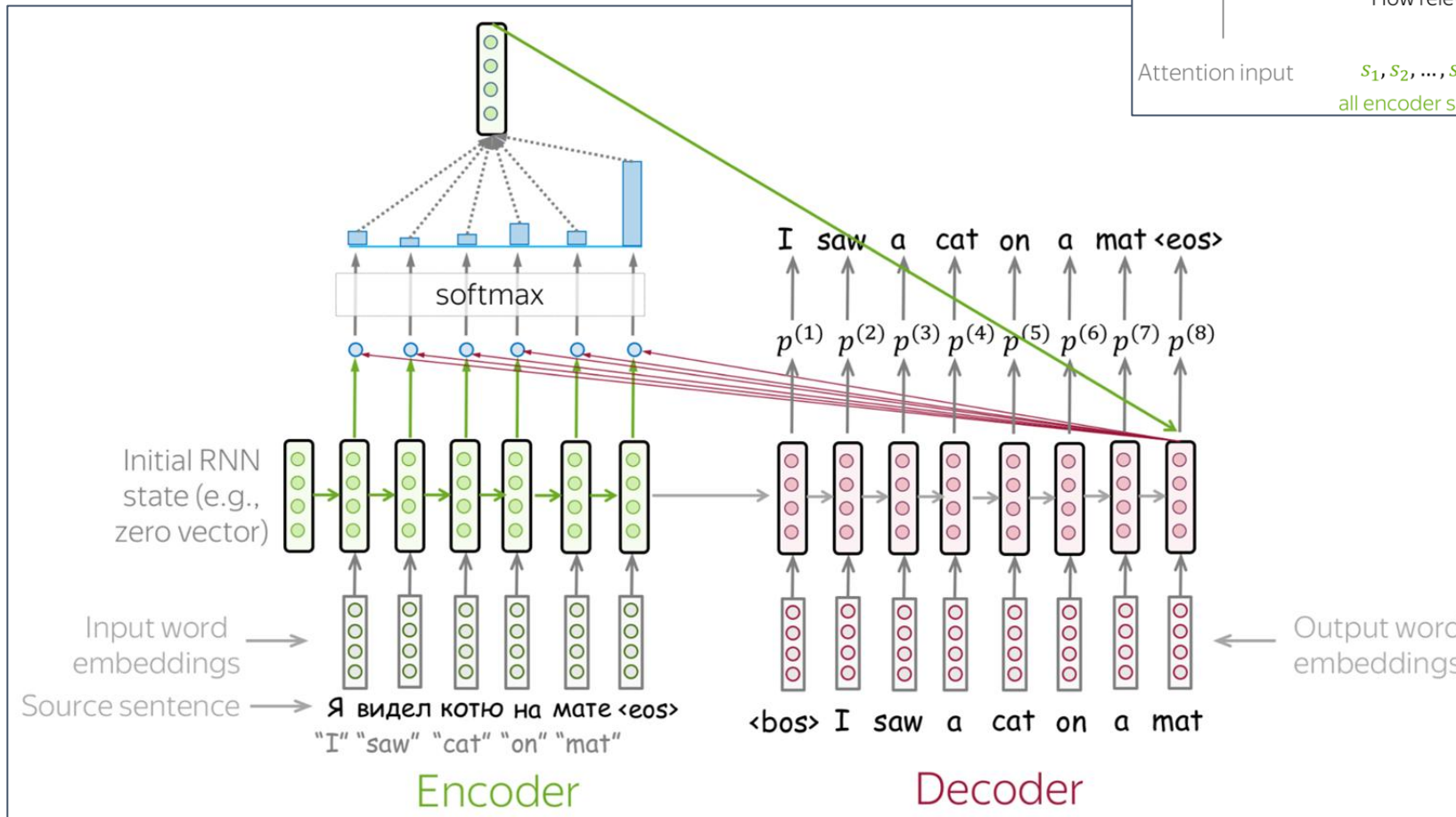
Attention mechanism



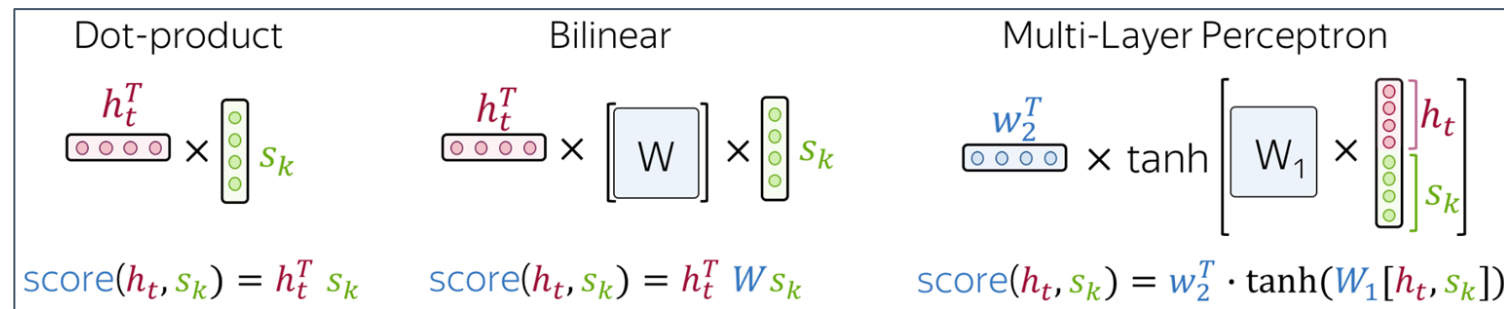
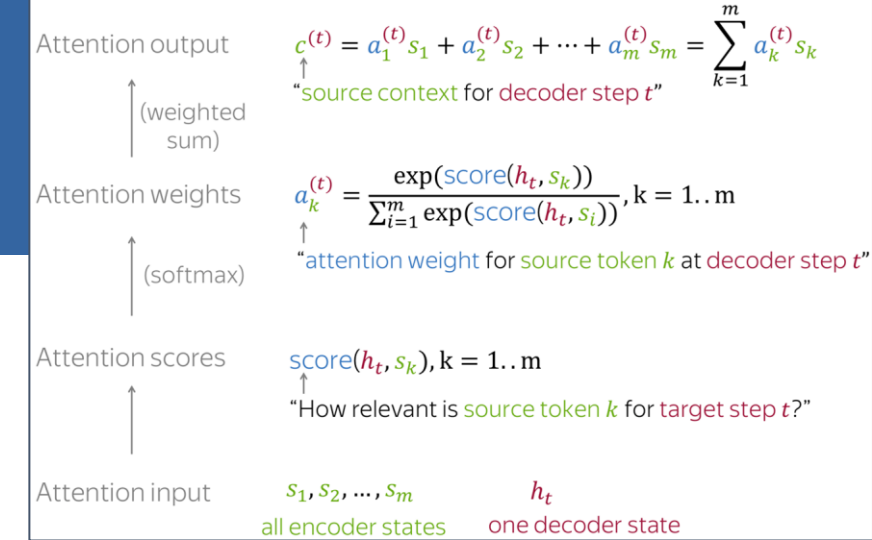
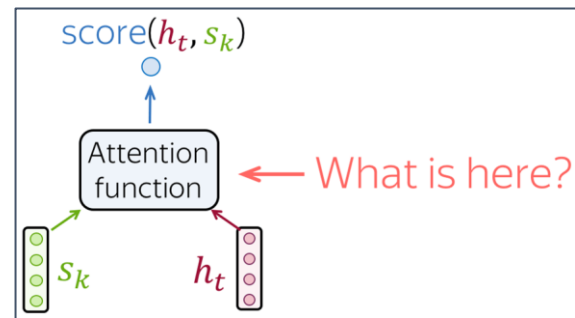
Attention mechanism



Attention mechanism

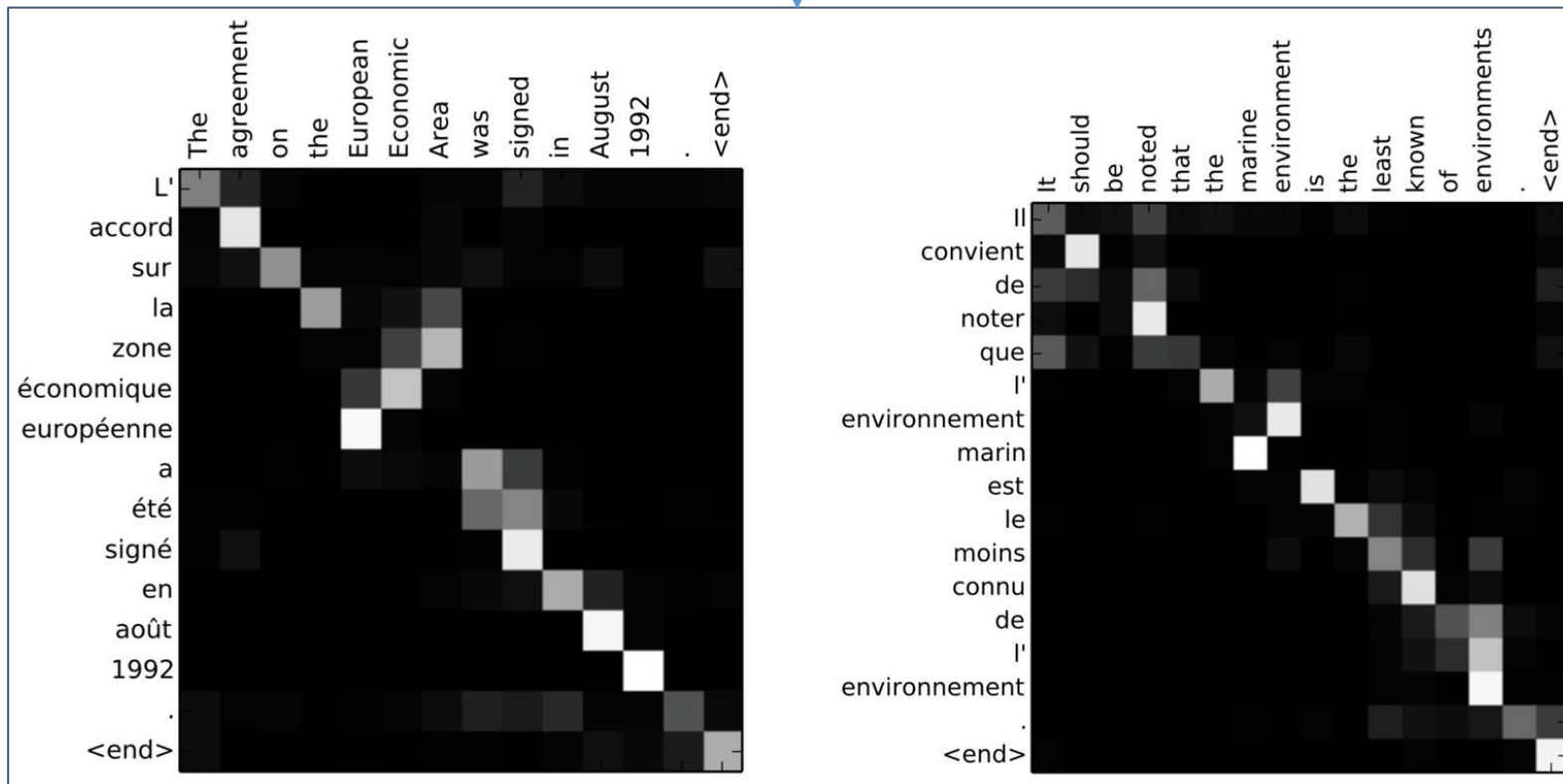
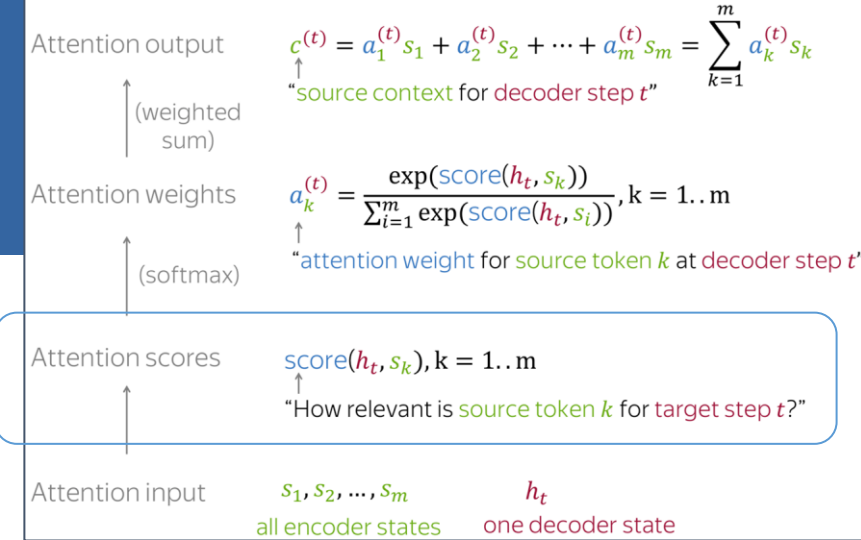


Attention mechanism



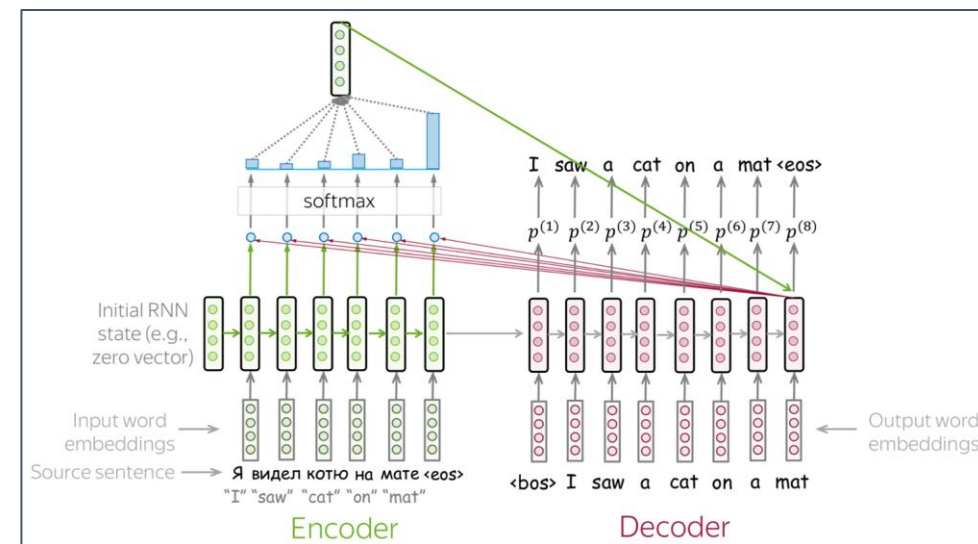
Various attention functions

Attention mechanism



Attention mechanism

- How to train?
 - At each moment in time, the model predicts a probability distribution over our vocabulary given the source context and previously predicted target tokens
 - Minimize the cross entropy between the target and model distribution



$s_1, s_2 \dots s_m = \text{Encoder}(\text{Emb}(\text{Я видел котю на мате}))$ - source context

$$p_{\text{model}}^{(t)} \in \mathbb{R}^{\text{vocab_size}}$$

$$\begin{aligned} p_{\text{model}}^{(t)}(* \parallel \hat{y}_0, \hat{y}_1, \dots, \hat{y}_{t-1}, s_1, s_2 \dots s_m) &= \\ &= \text{Decoder}(\hat{y}_0, \hat{y}_1, \dots, \hat{y}_{t-1}, s_1, s_2 \dots s_m) \end{aligned}$$

Attention mechanism

- How to train?
 - Minimize the cross entropy between the target and model distribution
 - $\mathcal{L}_{ce} = -\sum_i p(i) \cdot \log q(i)$
 - $p(i)$: the ground truth distribution
 - $q(i)$: the predicted distribution

cat	0.1	0.9	0.05
dog	0.7	0.05	0.15
bird	0.2	0.05	0.8
	i=1	i=2	i=3

Model이 $q(i)$ 를 예측
→ $q(i) = \{0.1, 0.7, 0.2\}$

Vocab={cat, dog, bird}

i=1일 때, 정답이 dot

$$p(i) = [0, 1, 0]$$

$$\begin{aligned}\mathcal{L}_{ce} &= -\sum_i p(i) \cdot \log q(i) \\ &= -(0 \cdot \log 0.1 + 1 \cdot \log 0.7 + 0 \cdot \log 0.2) \\ &= -\log 0.7\end{aligned}$$

Cross Entropy는 정보량의 기댓값:

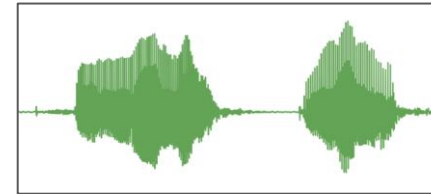
→ 즉, 두 분포의 분포가 다를 수록 정보량이 많다 (새로운 정보가 많다)

→ 두 분포가 다를 수록 Cross Entropy의 크기는 커진다.

Alignment problem, Speech case

- **Task:** Automatic Speech Recognition (ASR)

- We want to transform $x_1, \dots, x_T$ to $y_1, \dots, y_U$ where $T \neq U$ usually



hello world

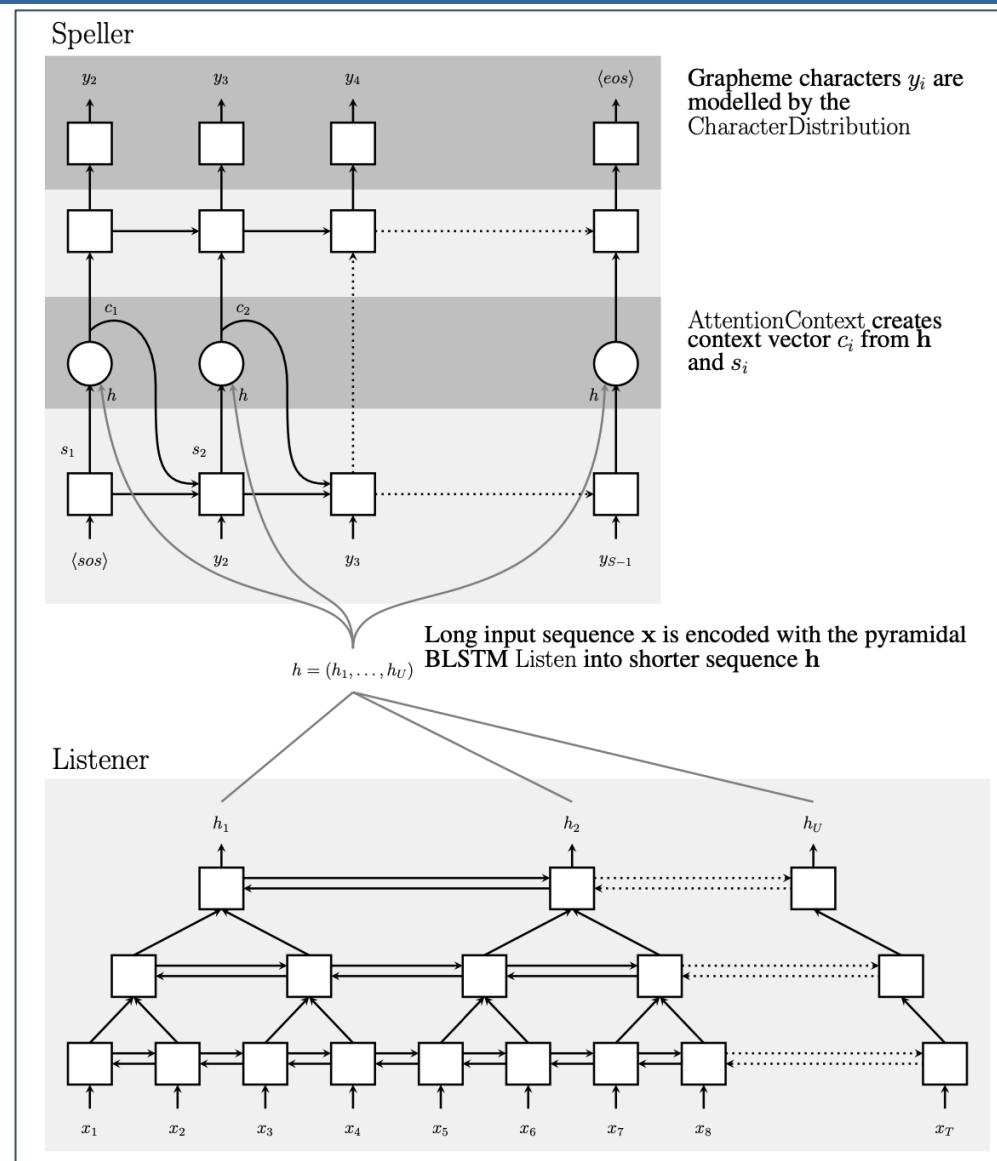
$s_1, s_2 \dots s_m = \text{AudioEncoder}(\text{Spec}(\text{wav}))$ - source context

$$p_{\text{model}}^{(t)} \in \mathbb{R}^{\text{vocab_size}}$$

$$p_{\text{model}}^{(t)} (* \parallel \hat{y}_0, \hat{y}_1, \dots, \hat{y}_{t-1}, s_1, s_2 \dots s_m) = \\ = \text{Decoder}(\hat{y}_0, \hat{y}_1, \dots, \hat{y}_{t-1}, s_1, s_2 \dots s_m)$$

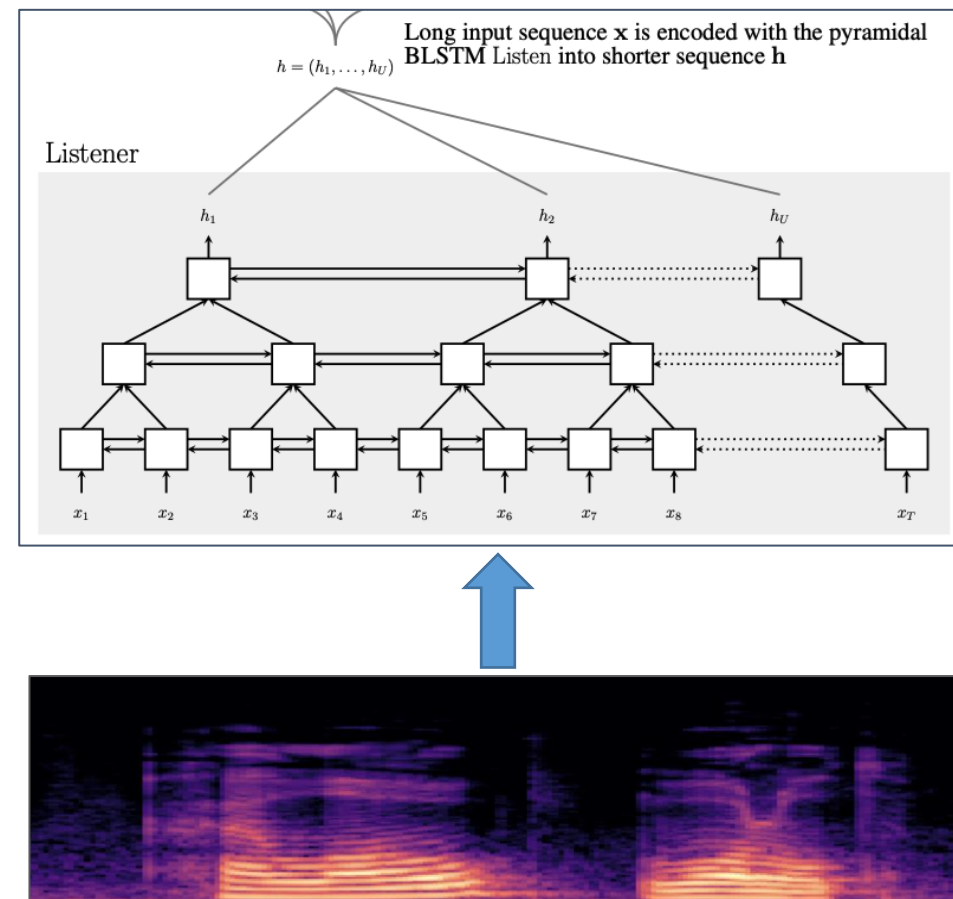
Listen, Attend and Spell (LAS)

- **Listen, Attend and Spell**, William Chan et al., Carnegie Mellon University, Google Brain, 2015
 - The network produces character sequences without making any independence assumptions between the characters.
- Modules:
 - Listener - encodes information about the input audio
 - Attend - tells Speller what parts of input are relevant during the current decoding step
 - Speller- decodes the latent representation into transcription



Listen, Attend and Spell (LAS)

- In fact, we can use arbitrary encoder (for example Conformer), but that was in 2015
- Thus, we use LSTM in the encoder and a pyramid structure for subsampling since we need to reduce the time dimension:
 - RNN can't handle very long sequences
 - Reduce the time dimension for Attend for selecting the most relevant information
 - Better convergence in much shorter time
- 8x time reduction (4x in the image)



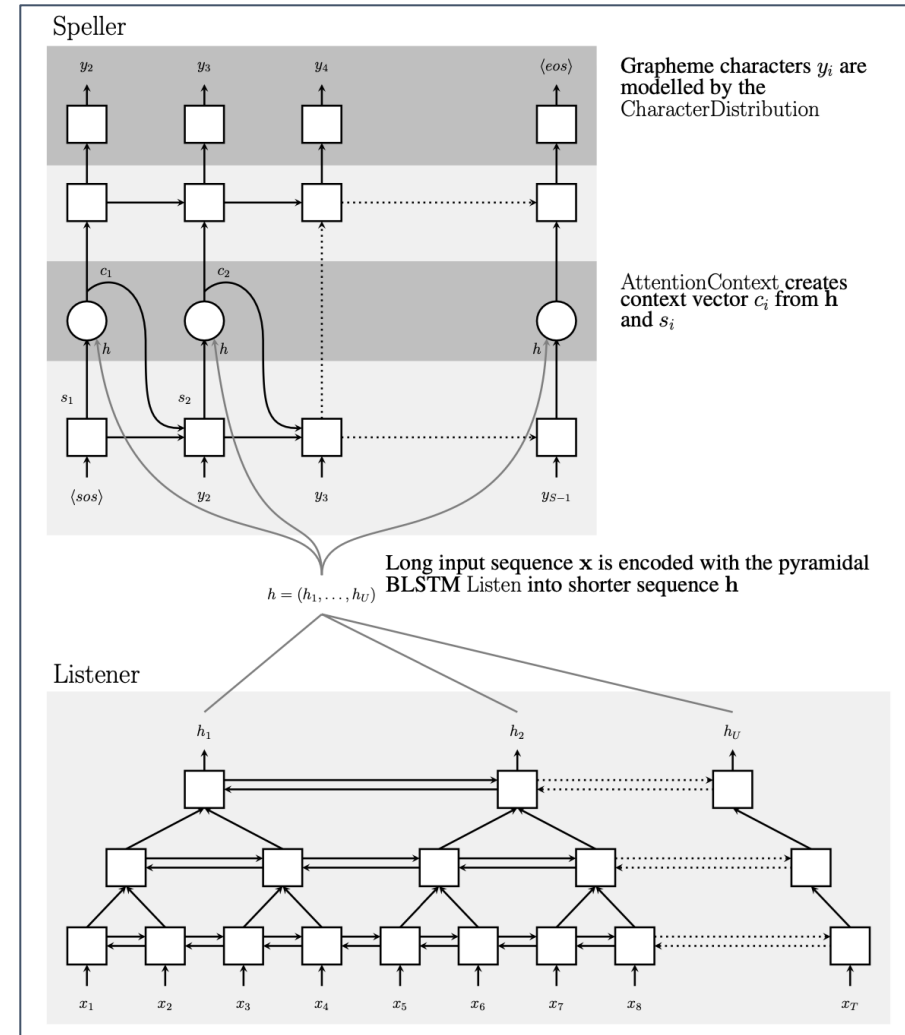
Listen, Attend and Spell (LAS)

- by argmax (greedy decoding):

$$p_{\text{model}}^{(t)} (* \parallel \hat{y}_0, \hat{y}_1, \dots, \hat{y}_{t-1}, h_1, h_2 \dots h_U) = \\ = \text{Decoder}(\hat{y}_0, \hat{y}_1, \dots, \hat{y}_{t-1}, h_1, h_2 \dots h_U)$$

$$\hat{y}_1 = \arg \max p_{\text{model}}^{(1)} (* \parallel y_0, h_1, h_2 \dots h_U) \text{ (} y_0 \text{ bos emb)}$$
$$\hat{y}_2 = \arg \max p_{\text{model}}^{(2)} (* \parallel y_0, \hat{y}_1, h_1, h_2 \dots h_U)$$
$$\dots$$

- by usual beam search:
 - Expand beam
 - Truncate beam



Listen, Attend and Spell (LAS)

- LAS quality

Method	Year	WER (test -clean)	WER (test-oth er)
Human	~ 200 000 b.c.	5.83	12.69
Deep Speech 2	2015	5.15	12.73
LAS	2015	-----	-----
LAS without S pecAugment*	2019	3.2	9.8

Listen, Attend and Spell (LAS)

- CTC vs. LAS Inference

Method	CTC	LAS
Streaming	yes	no
Context	no	yes
Argmax Complexity	$O(\text{enc} + \text{dec})$	$O(\text{enc} + T * \text{dec})$
Beam Search Complexity	$O(\text{enc} + \text{dec} + T * \text{bs})$	$O(\text{enc} + T * \text{bs} * \text{dec})$

Listen, Attend and Spell (LAS)

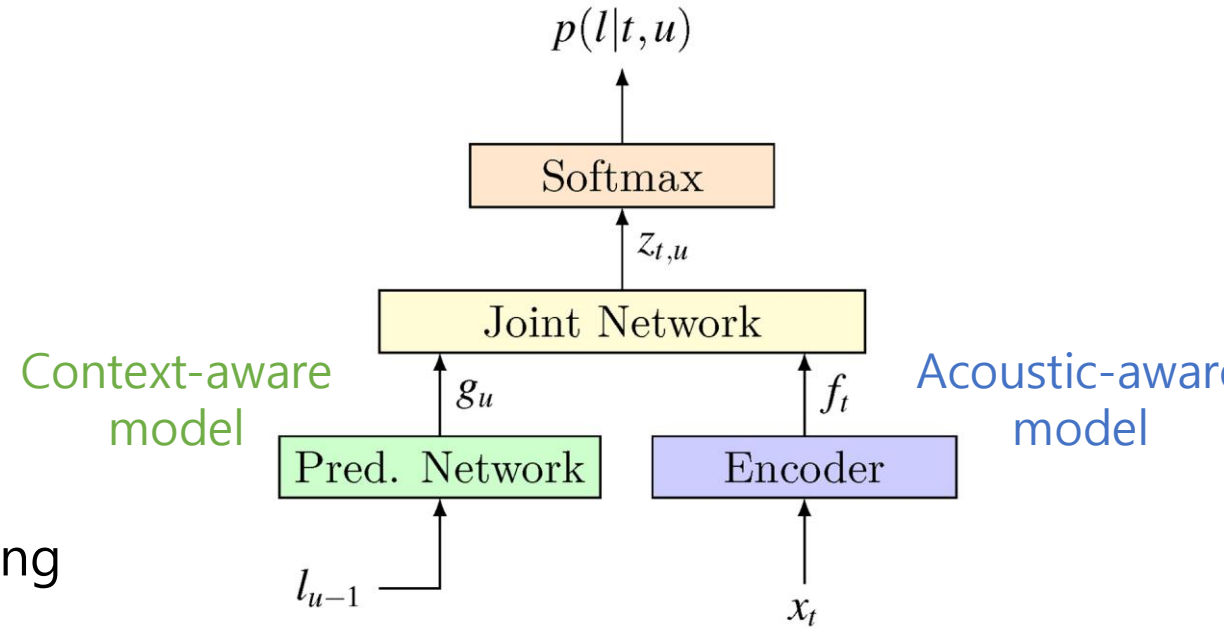
- Can do better?

Method	CTC	LAS	?
Streaming	yes	no	yes
Context	no	yes	yes
Argmax Complexity	$O(\text{enc} + \text{dec})$	$O(\text{enc} + T * \text{dec})$	?
Beam Search Complexity	$O(\text{enc} + \text{dec} + T * \text{bs})$	$O(\text{enc} + T * \text{bs} * \text{dec})$	?

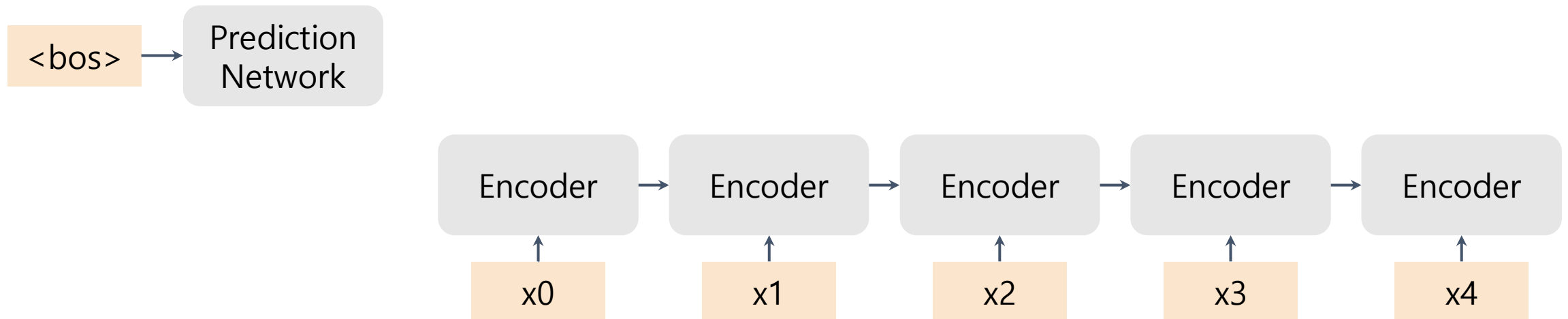
RNN-Transducer (RNN-t)

- **Sequence Transduction with Recurrent Neural Networks**, Alex Graves, University of Toronto, 2012

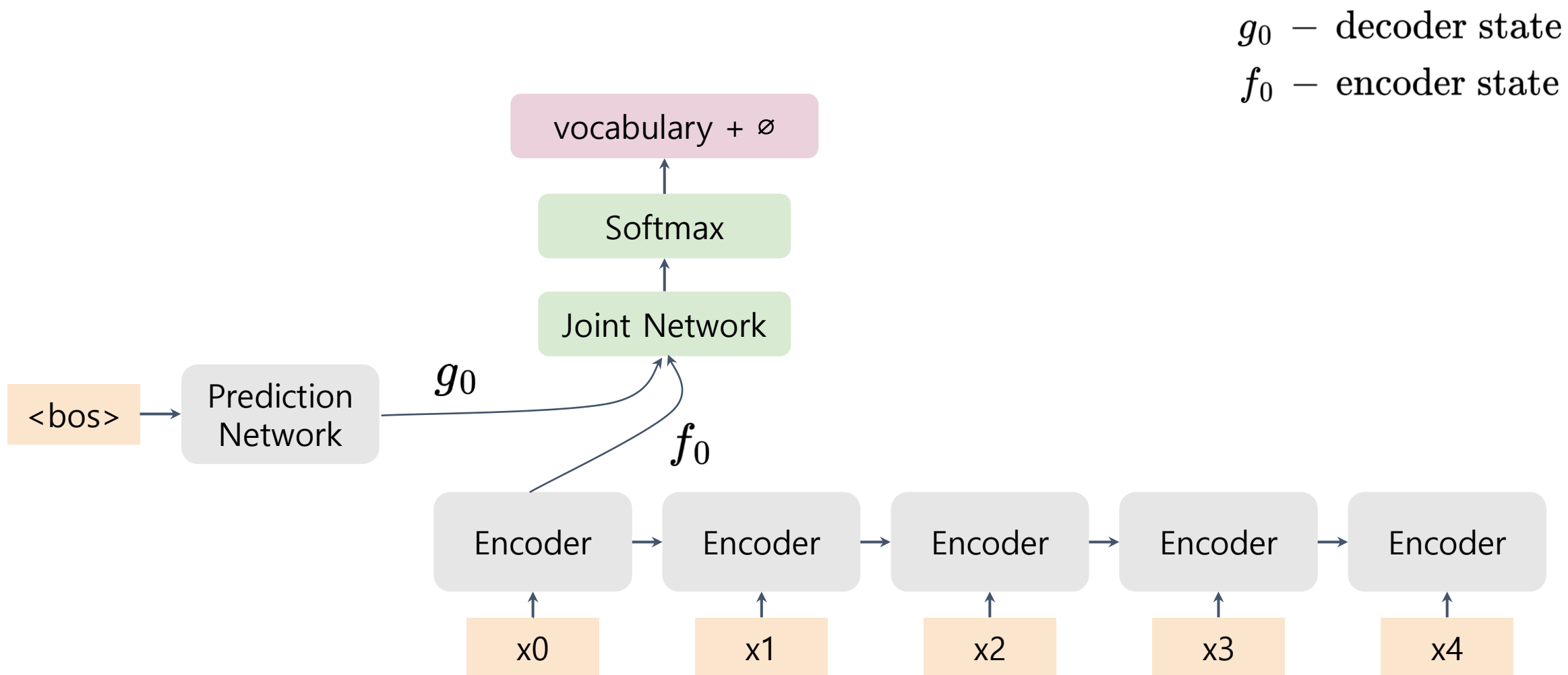
- Motivation or we want:
 - A context-dependent model
 - A streamable model
- Architecture:
 - **Audio encoder** - for acoustic features
 - **Prediction network** - for text processing
 - **Joint network** - for predicting with audio-language context



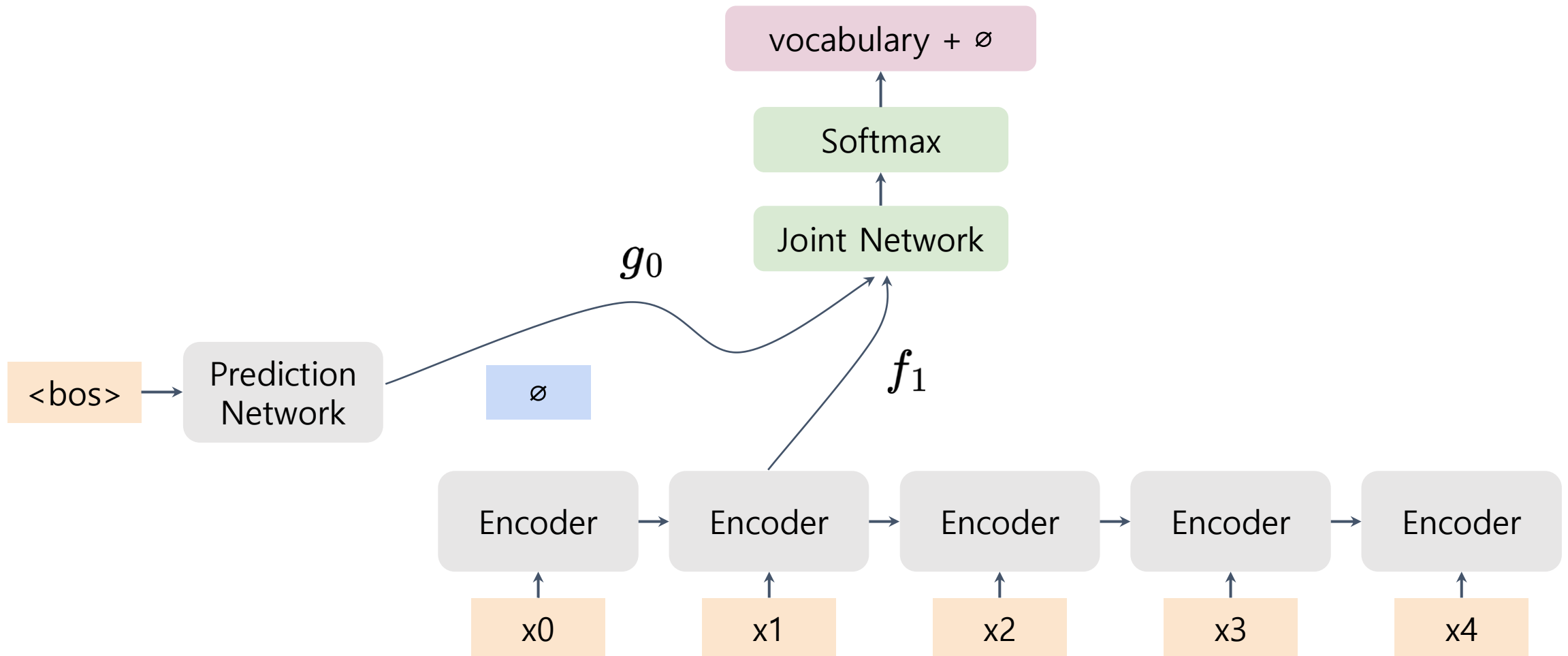
RNN-t Inference



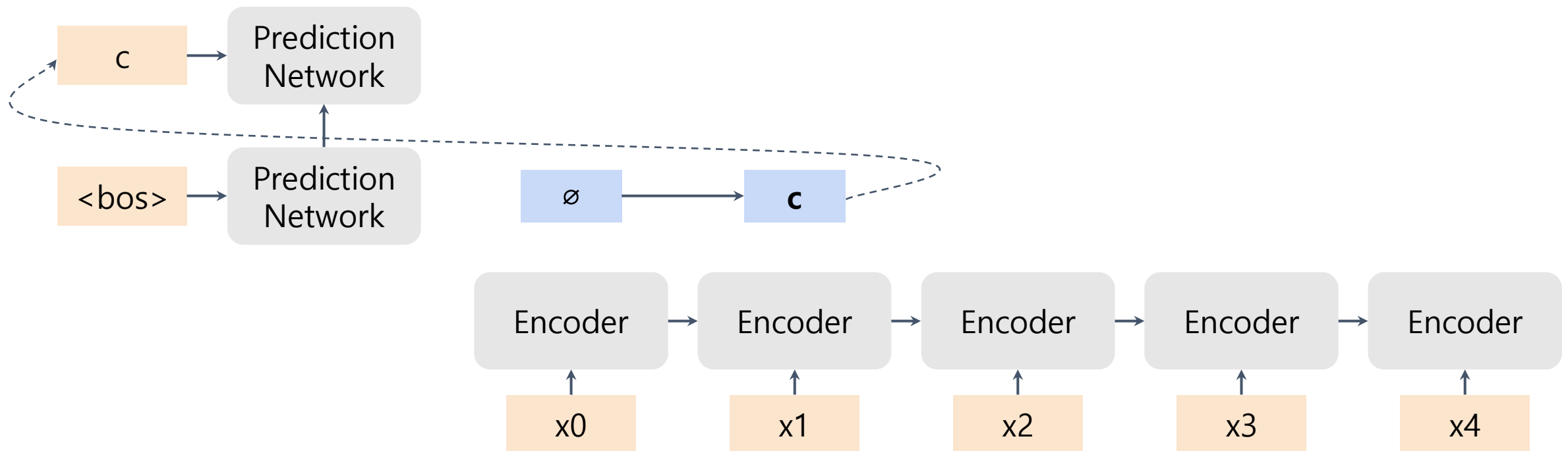
RNN-t Inference



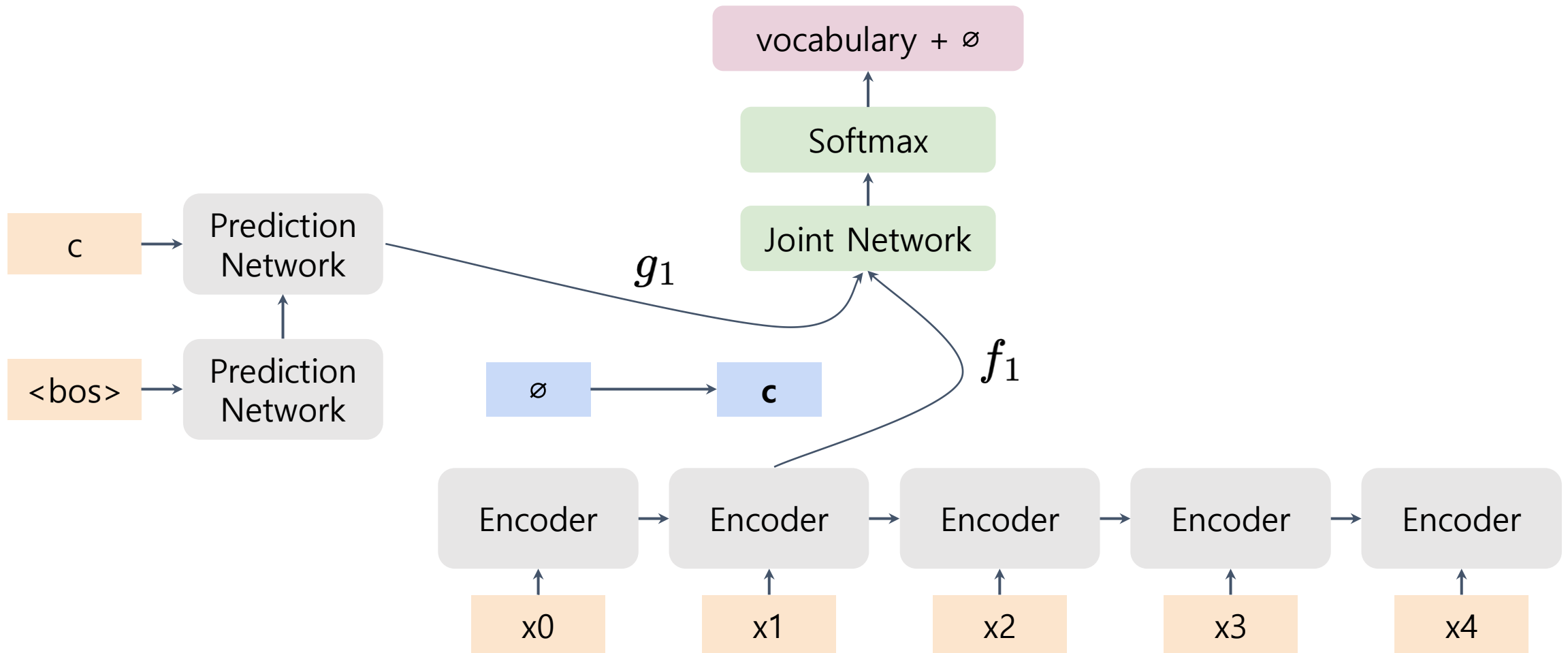
RNN-t Inference



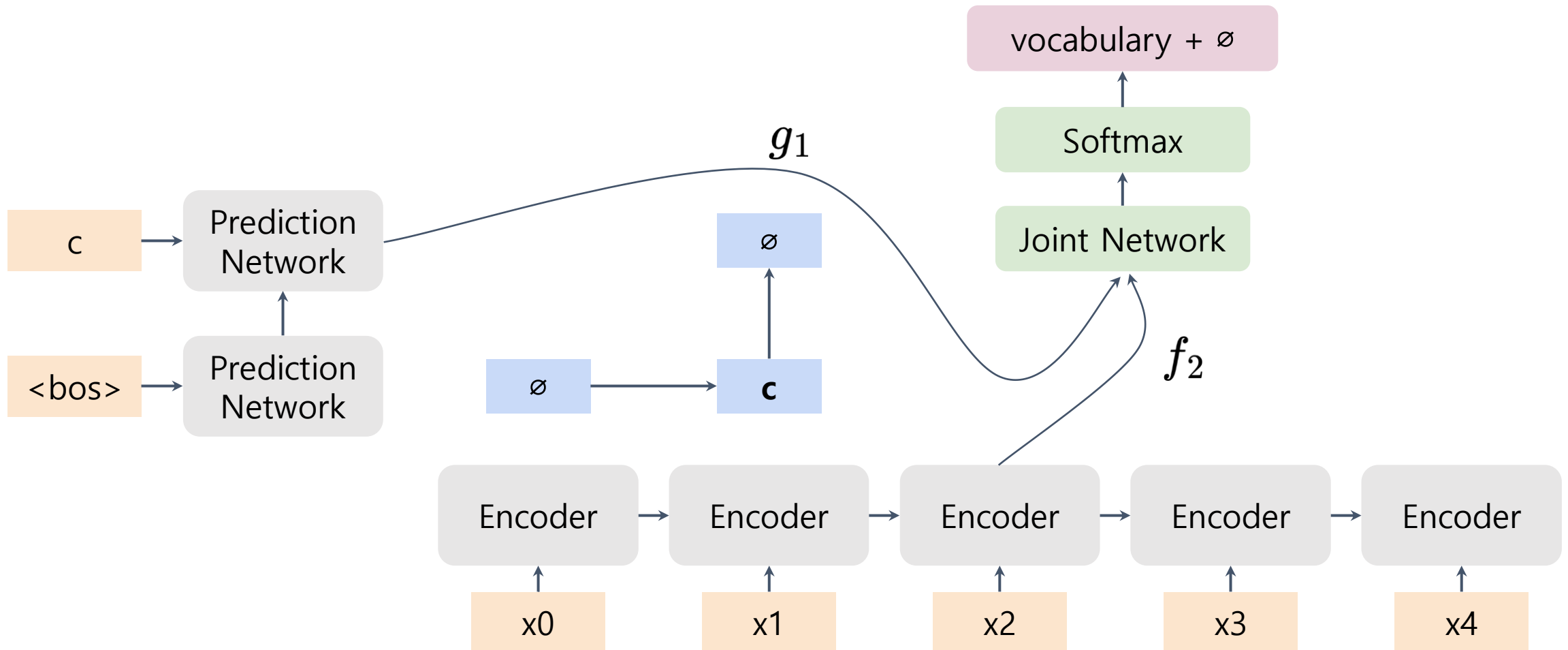
RNN-t Inference



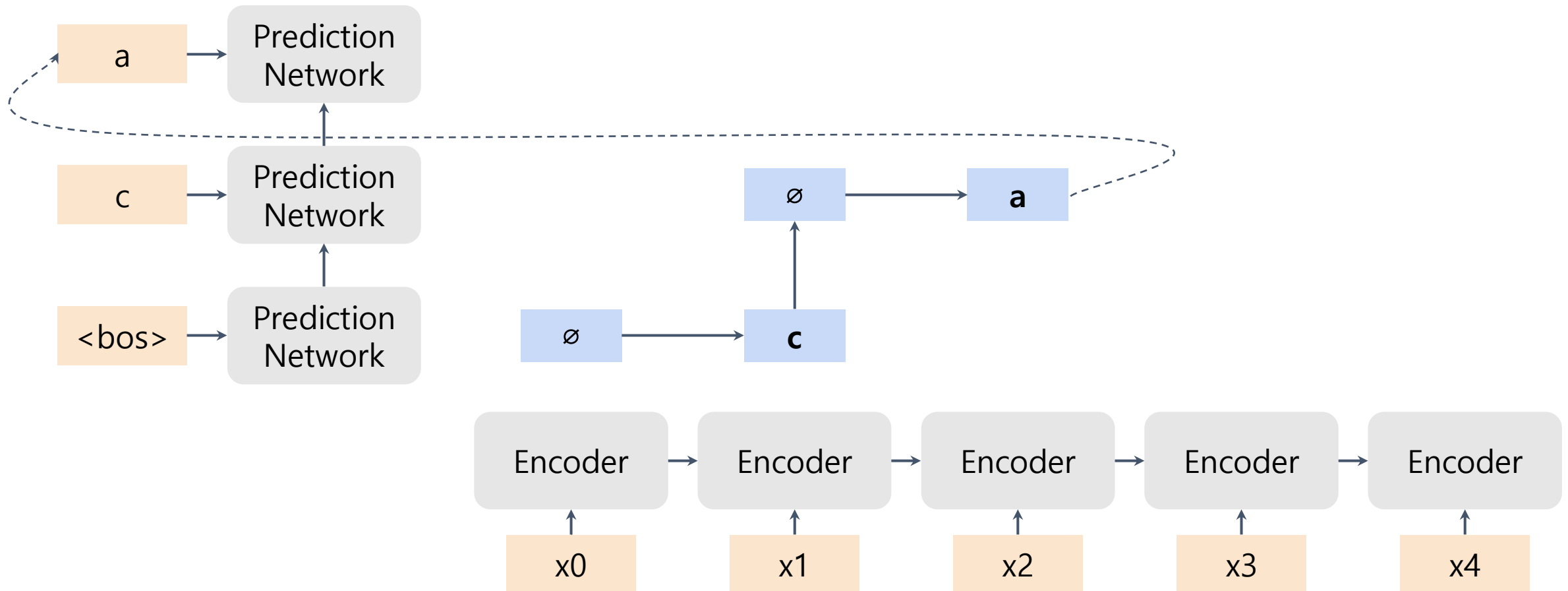
RNN-t Inference



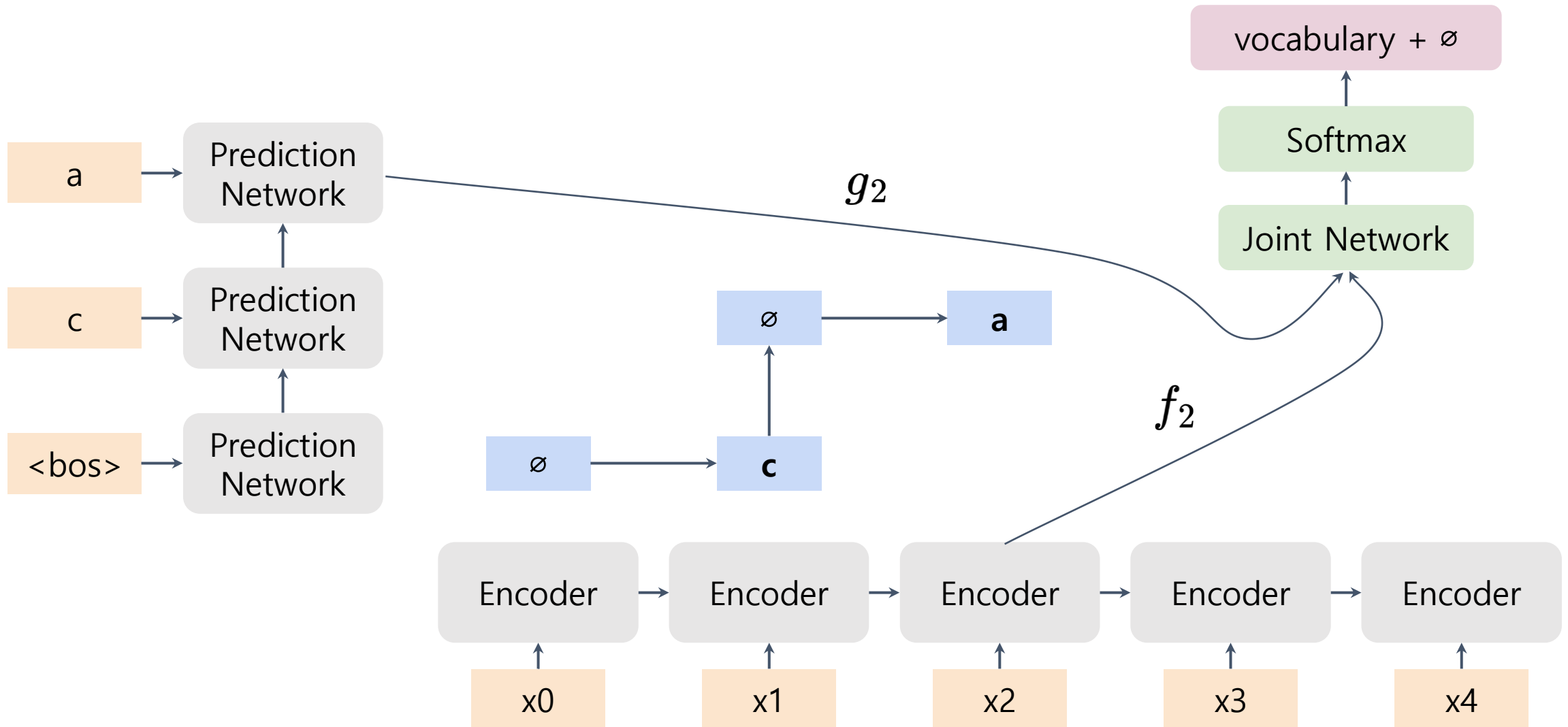
RNN-t Inference



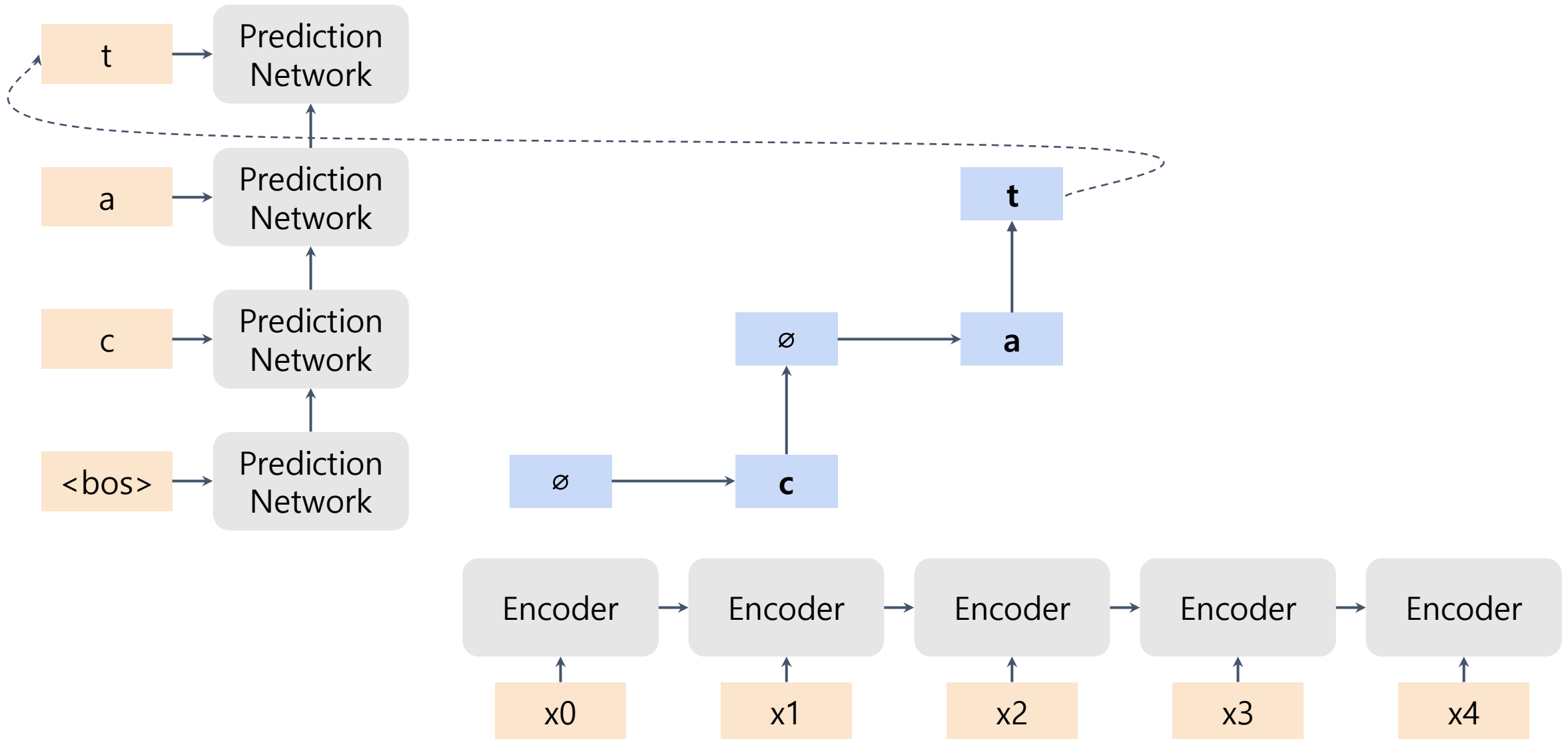
RNN-t Inference



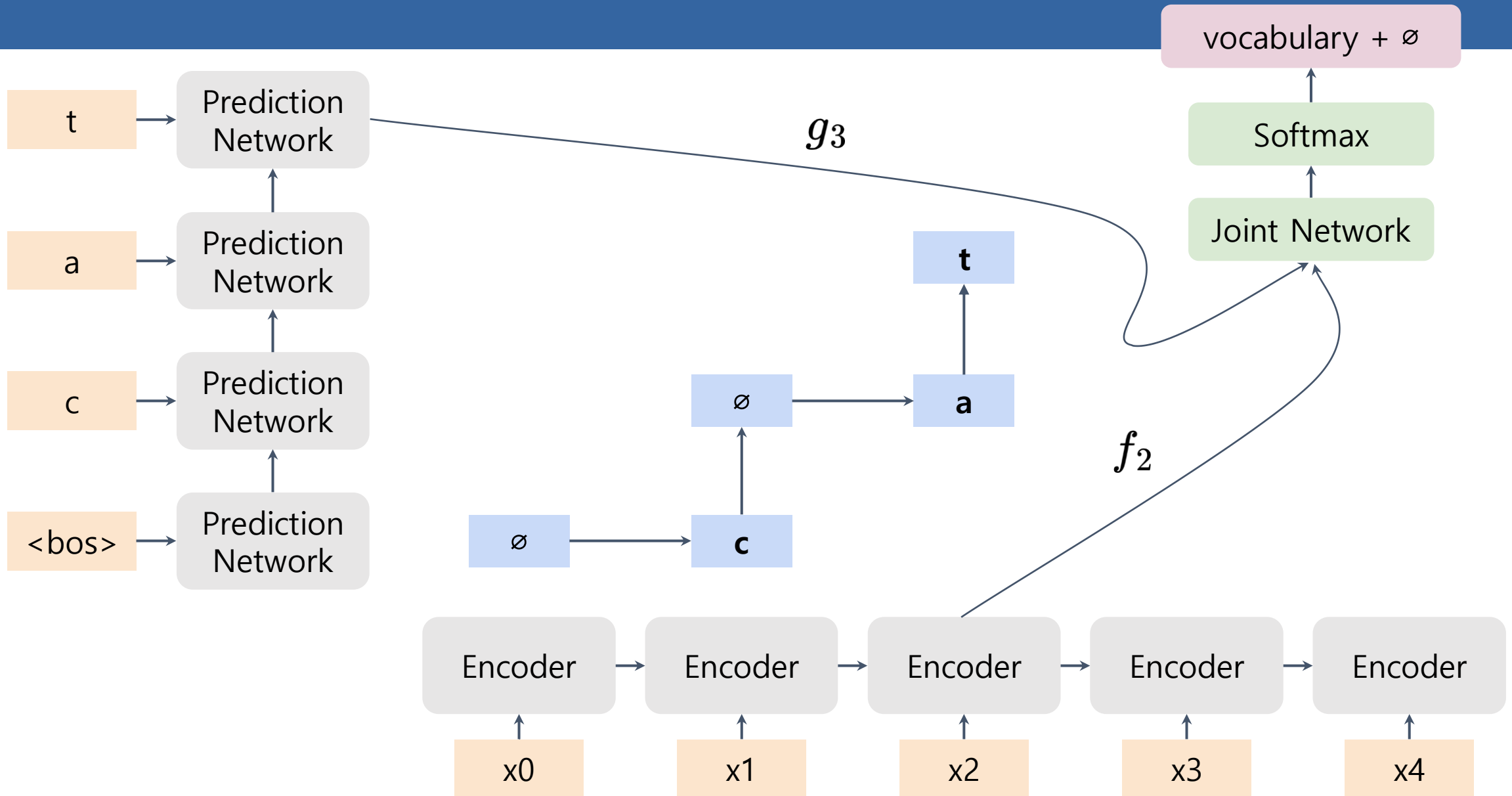
RNN-t Inference



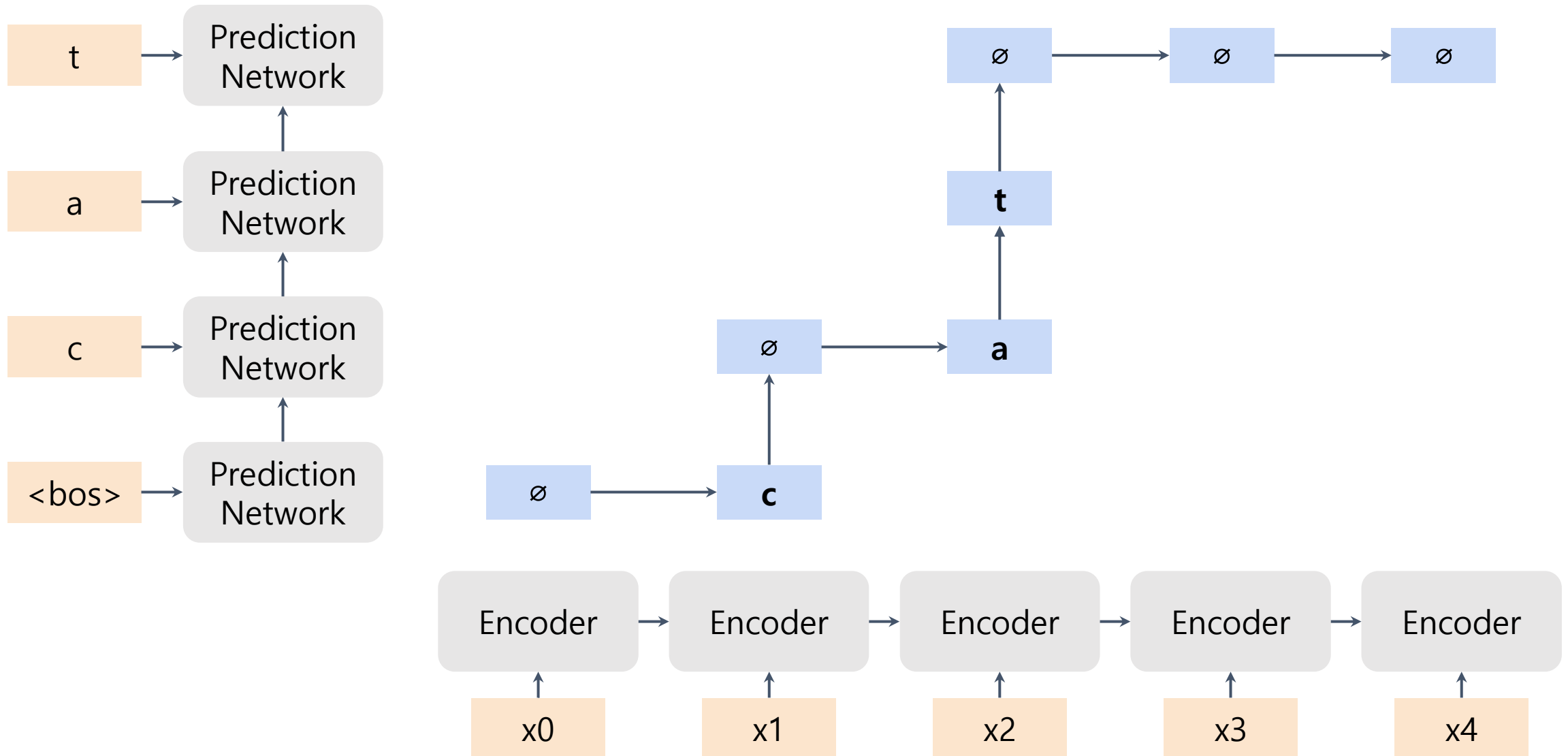
RNN-t Inference



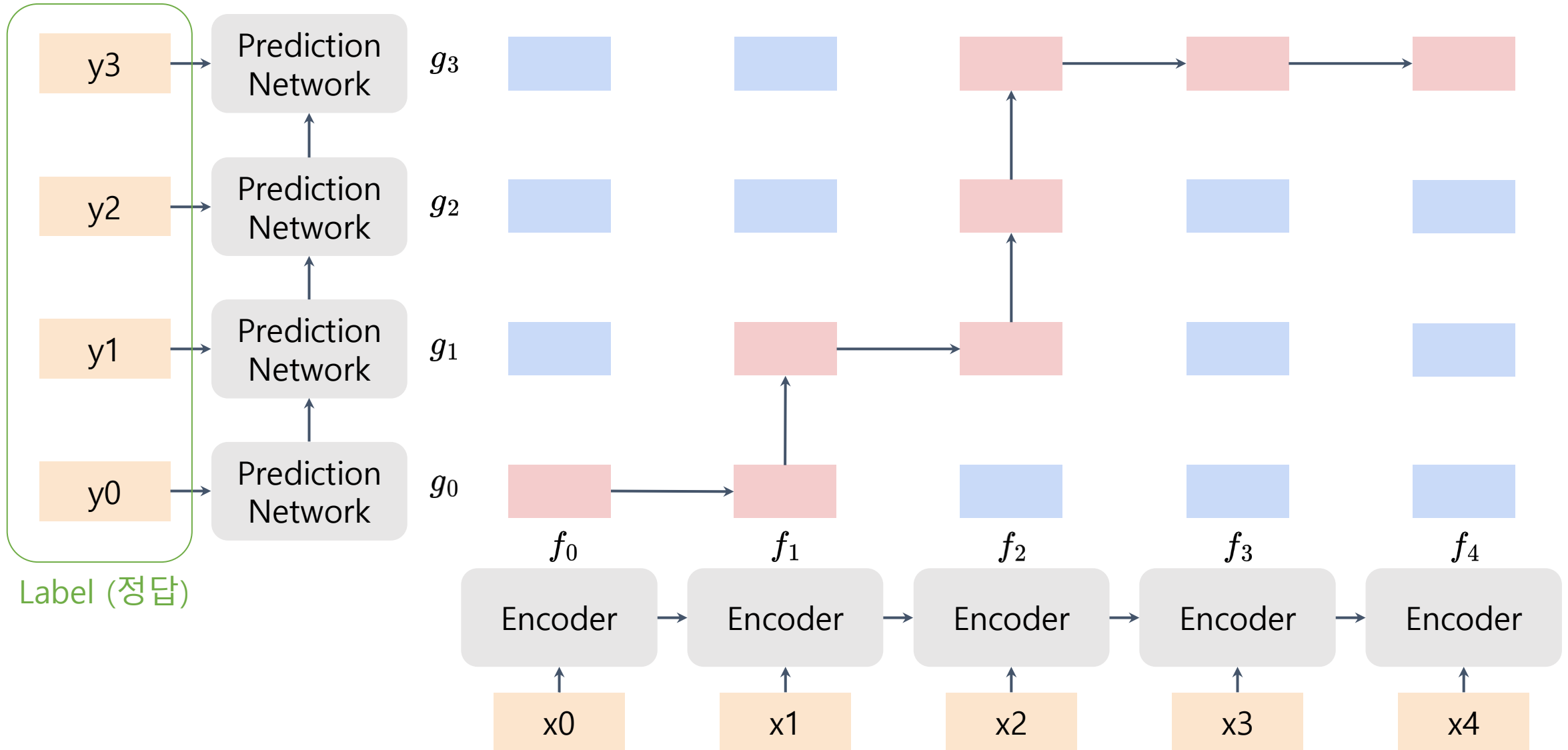
RNN-t Inference



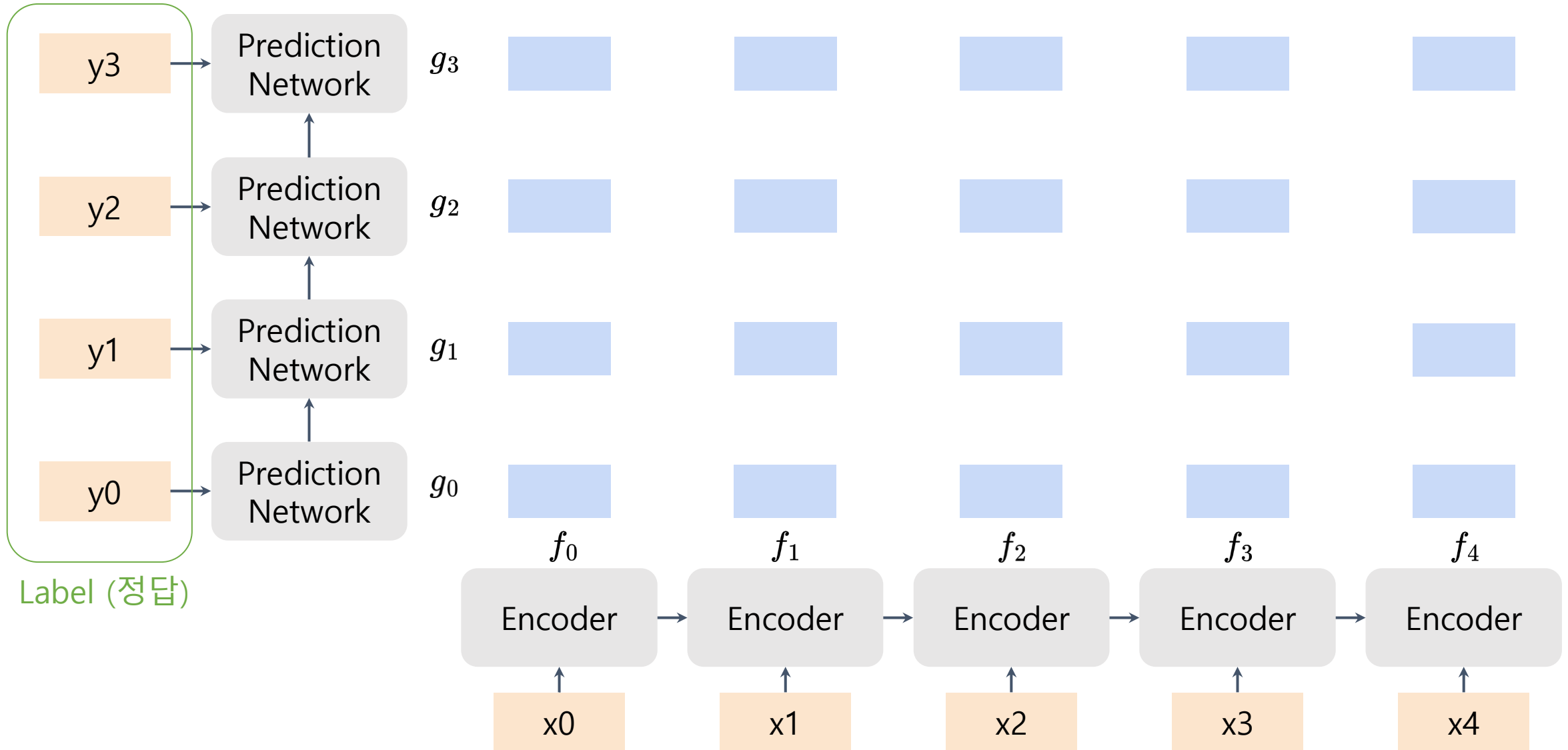
RNN-t Inference



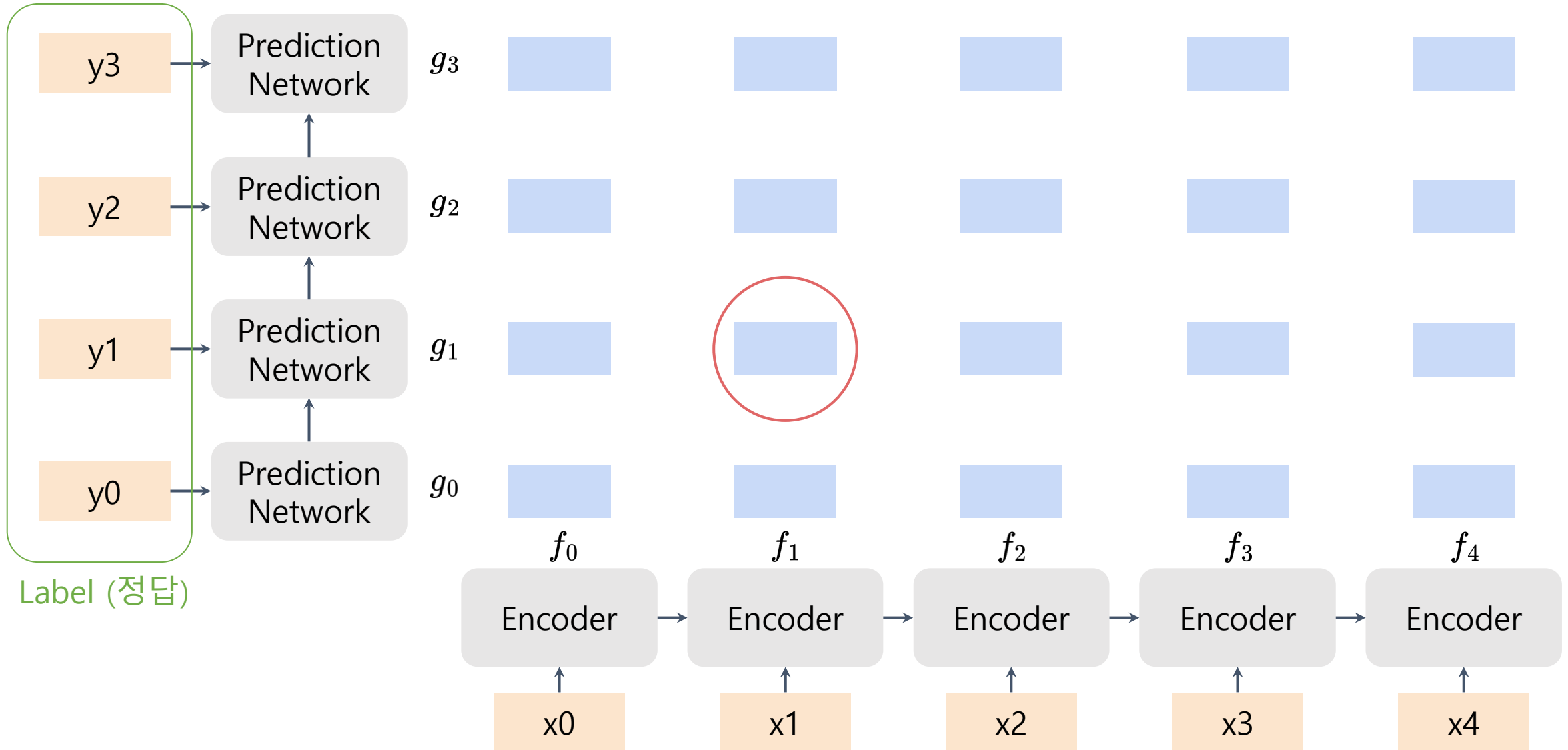
RNN-t train



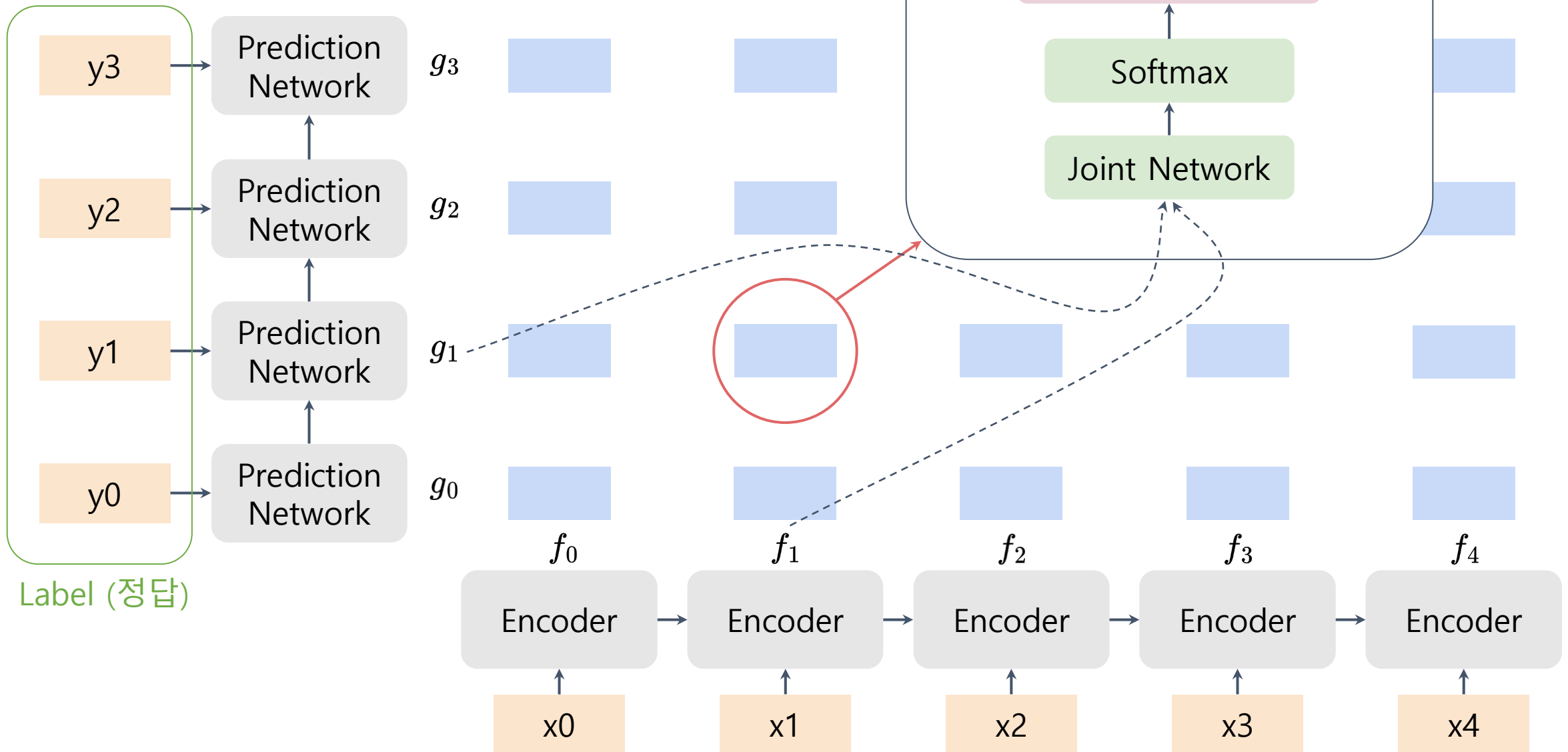
RNN-t train



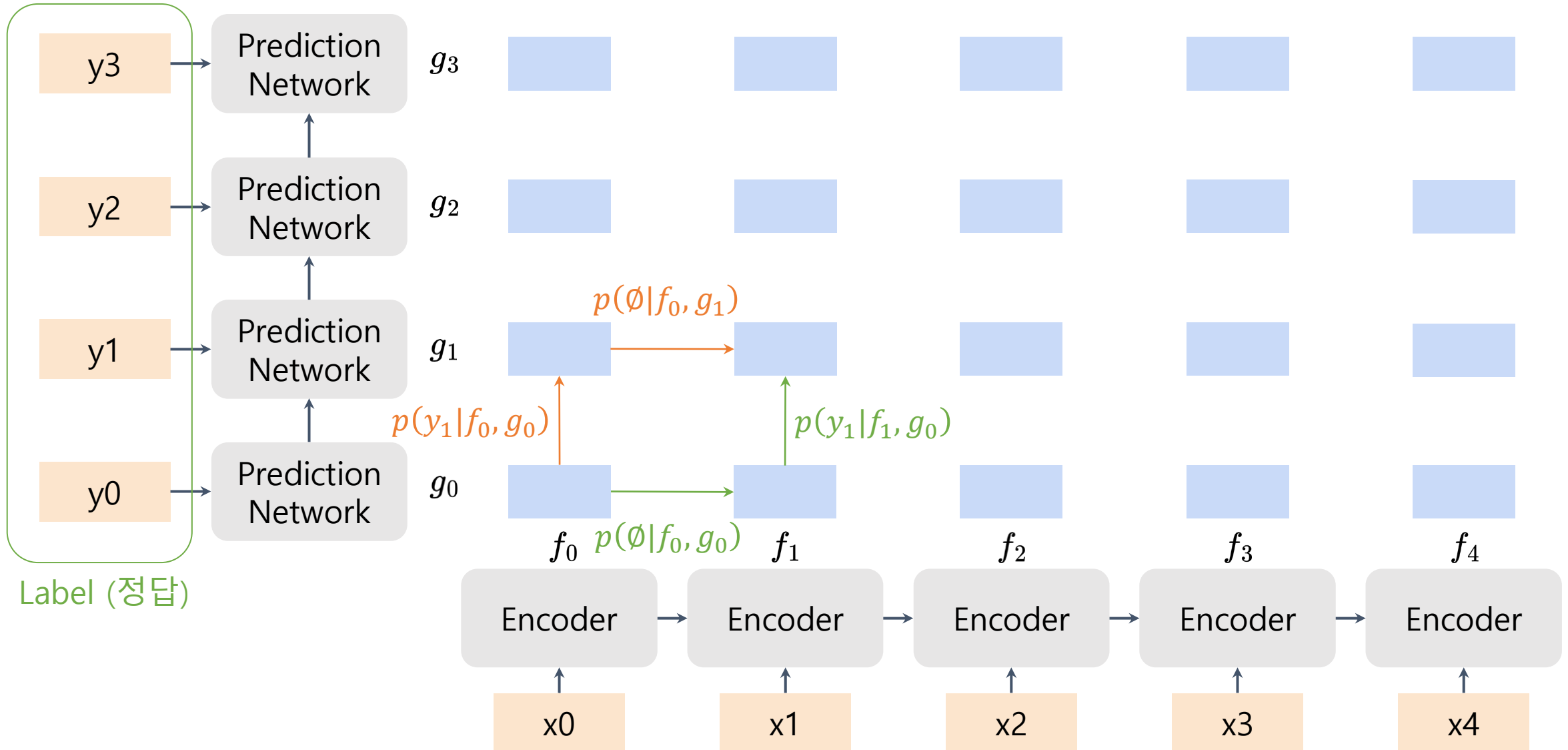
RNN-t train



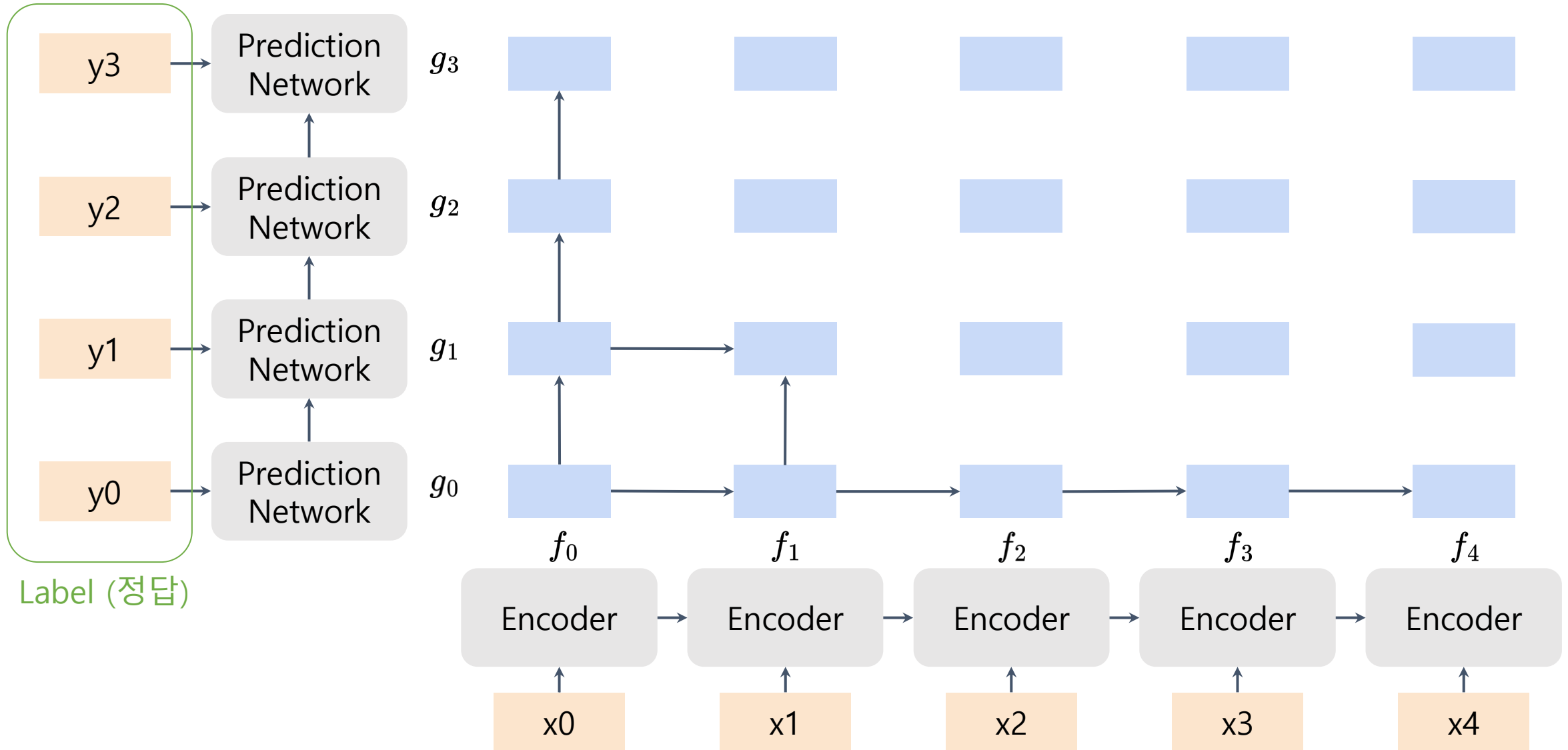
RNN-t train



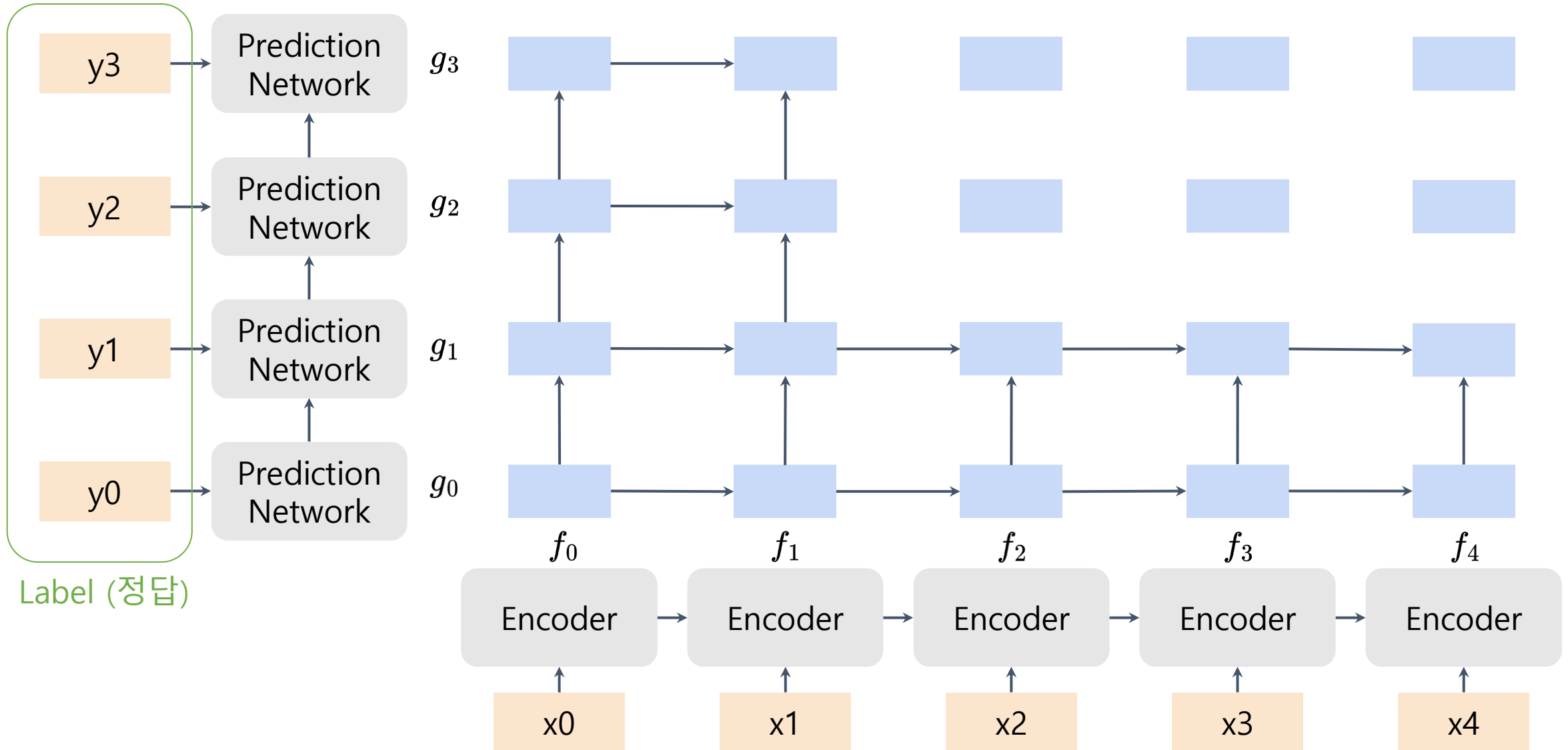
RNN-t train



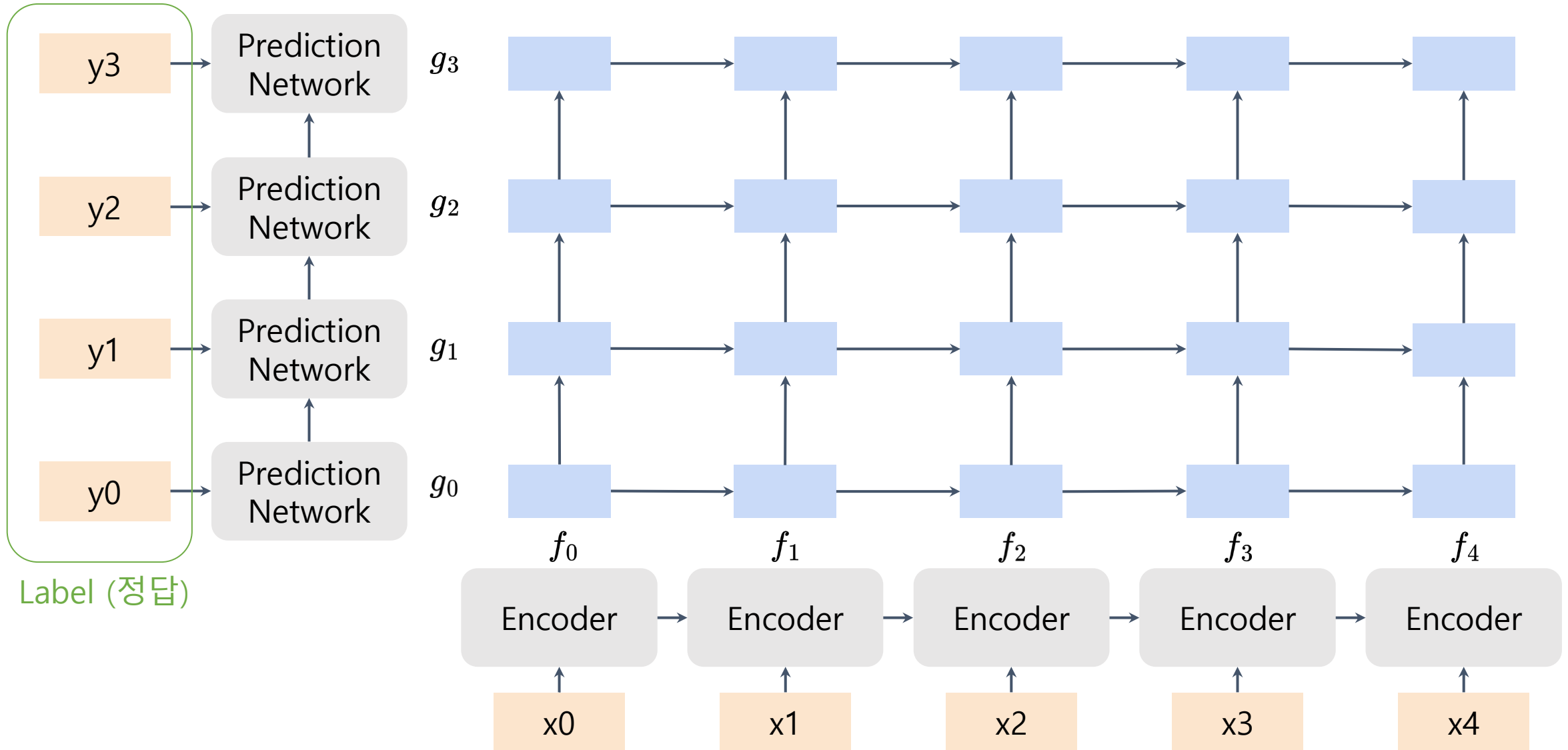
RNN-t train



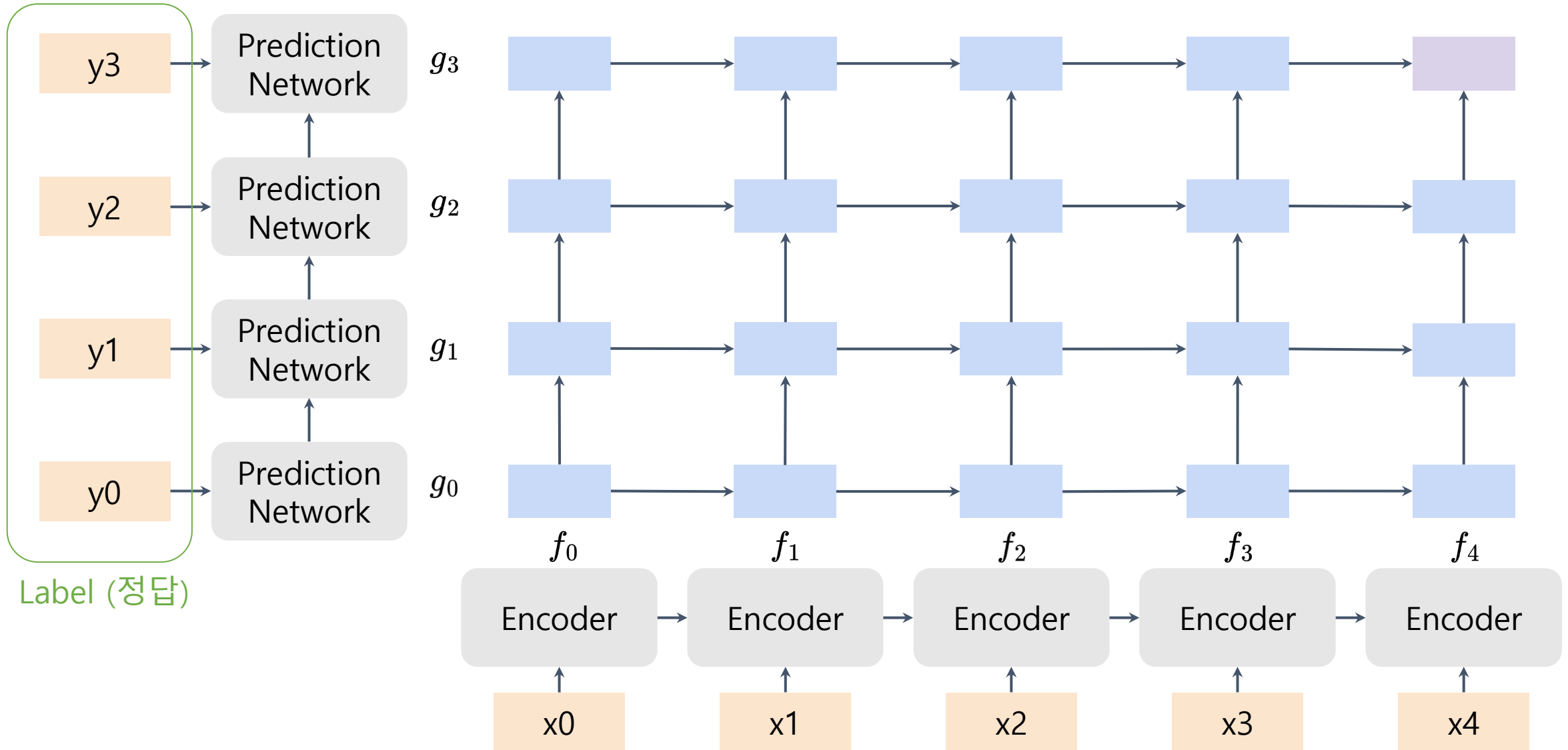
RNN-t train



RNN-t train



RNN-t train



RNN-t properties

- streamable and context dependent
- produce alignment (weak)
- it is difficult to compute efficiently
 - calculation of the joint module is hard
BS = 32 ; T (after 2x stride) = 800, U (with character encoding) = 400-450 tokens, V = 28. Let the hidden dimension of the Joint model be 640.
 - Hidden = $32 \times 800 \times 450 \times 640 \times 4 = 29.49 \text{ Gb (fp32)}$
Joint = $32 \times 800 \times 450 \times 28 \times 4 = 1.290 \text{ Gb (fp32)}$
 - This is just for the forward pass. We need to double this memory to store gradients.
 - BS=32 ; T (after 8x stride) = 200, U (with sub-word encoding) = 100-180 tokens, Vocabulary size V = 1024.
 - Hidden = $32 \times 200 \times 150 \times 640 \times 4 = 2.45 \text{ Gb (fp32)}$
Joint = $32 \times 200 \times 150 \times 1024 \times 4 = 3.93 \text{ Gb (fp32)}$

RNN-t Quality

Method	Year	WER (test -clean)	WER (test-oth er)
Human	~ 200 000 b.c.	5.83	12.69
Deep Speech 2	2015	5.15	12.73
LAS	2015	-----	-----
LAS without S pecAugment*	2019	3.2	9.8
RNN-t	2020	2.1	4.3

*<https://arxiv.org/abs/1904.08779>

CTC vs. LAS vs. RNN-t

Method	CTC	LAS	RNN-t
Streaming	yes	no	yes
Context	no	yes	yes
Argmax Complexity	$O(\text{enc} + \text{dec})$	$O(\text{enc} + T * \text{dec})$	$O(\text{enc} + T * \text{dec})$
Beam Search Complexity	$O(\text{enc} + \text{dec} + T * \text{bs})$	$O(\text{enc} + T * \text{bs} * \text{dec})$	$O(\text{enc} + T * \text{bs} * \text{dec})$

DL-based ASR

- Language models (LM) - Refresher

- motivation:

- spelling of a word heavily depends on its context
 - LMs can add context dependency in CTC beam search
 - external language models typically saw more data (more than decoders in LAS, RNN-t)

- examples:

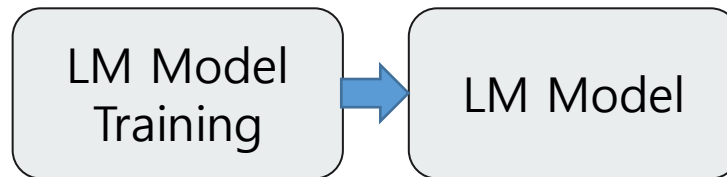
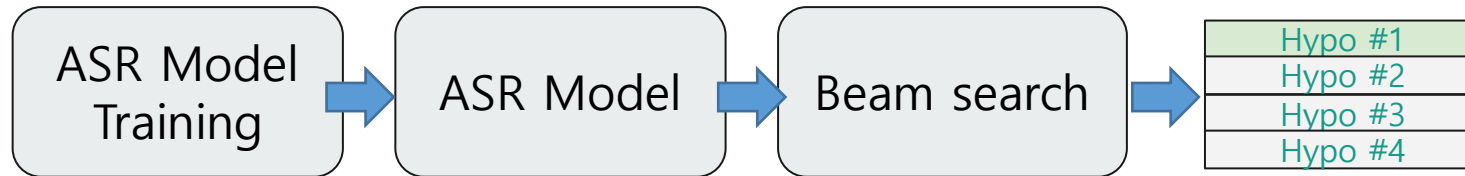
- simple: n-grams, Kneser–Ney smoothing and others
 - complex: neural networks
e.g: Bert, GPT, LLaMA

hypo 1: let's go **two** a movie (am score: 0.21)
hypo 2: let's go **to** a movie (am score: 0.19)
hypo 3: let's go **too** a movie (am score: 0.13)

$P(\text{let's go } \mathbf{two} \text{ a movie}) = 0.01$ (lm score)
 $P(\text{let's go } \mathbf{to} \text{ a movie}) = 0.6$ (lm score)

DL-based ASR

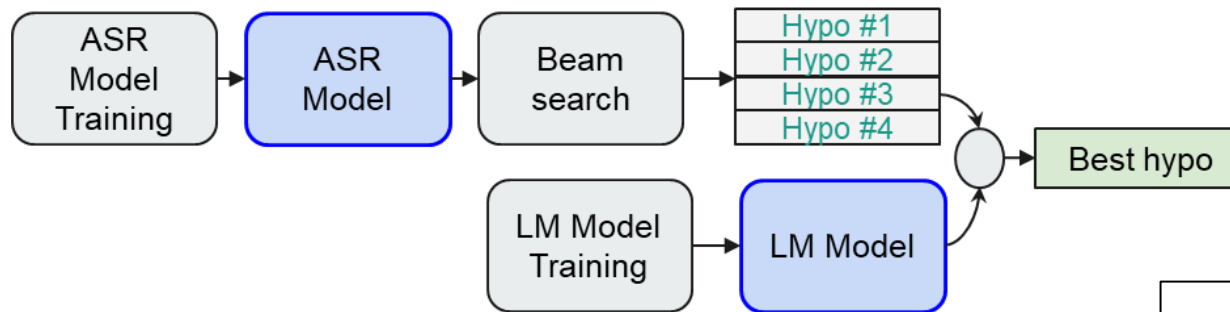
- Language models (LM) - Refresher



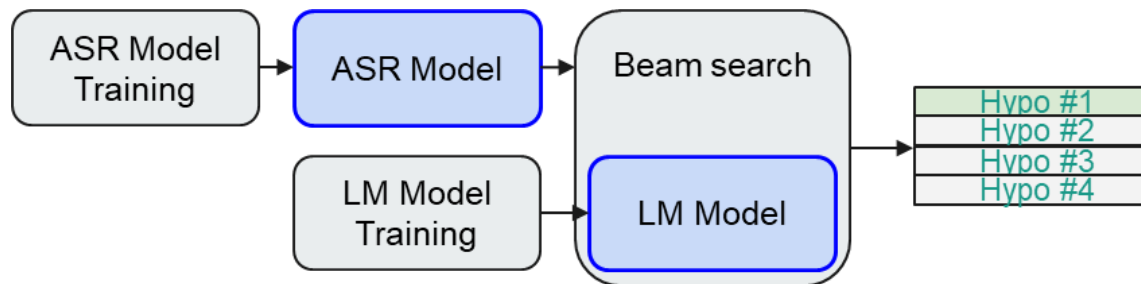
How to integrate LM?

DL-based ASR

- Language models (LM)
 - Second pass rescoring:



- Shallow fusion:

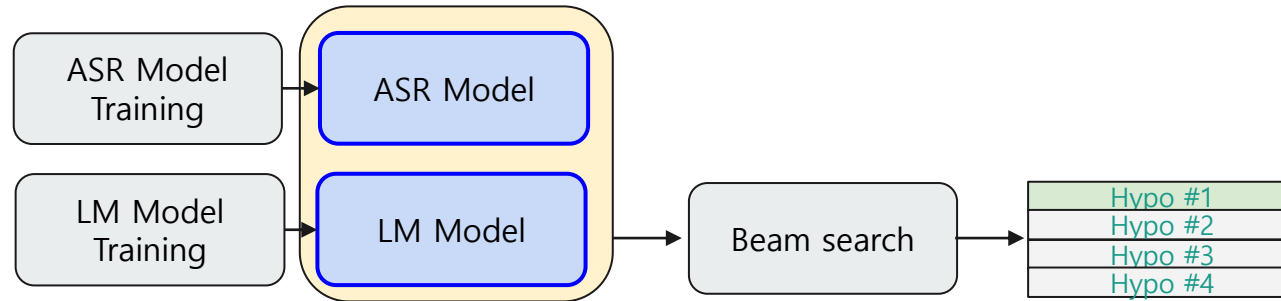


it's better to use:

- large model (ASR) for rescoring
- light model (ASR) for shallow fusion

DL-based ASR

- Language models (LM) with LAS and RNN-t
 - Deep Fusion:



- Cold Fusion:

